

Design of Scalable Resource Aware Neural Network Architectures for Object Segmentation using Depth Information

Please redefine the myshorttitle macro (may be empty):

```
\newcommand{\myshorttitle}{...}
```

Master's Thesis
of

Akash P. Parikh

KIT Department of Informatics
Institute for Anthropomatics and Robotics (IAR)
High Performance Humanoid Technologies Lab (H²T)

Duration: December 15th, 2020 — June 15th, 2020

Erklärung

Ich versichere hiermit, dass ich die Arbeit selbstständig verfasst habe, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, die wörtlich oder inhaltlich übernommenen Stellen als solche kenntlich gemacht habe und die Satzung des Karlsruher Instituts für Technologie zur Sicherung guter wissenschaftlicher Praxis beachtet habe.

Karlsruhe, den June 15th, 2020

Akash P. Parikh

Zusammenfassung

Handprothesen sind ein Segen für Amputierte. Das Haupthindernis bei der Verwendung von Handprothesen ist das natürliche Greifen des Objekts. Die herkömmliche myoelektrische Hand benötigt zum Zeitpunkt des Ergreifens eines Objekts zu viel Fokus. Diese kognitive Belastung zum Zeitpunkt des Greifens zu reduzieren, ist das aktuelle Ziel der Forschung. Die KIT-Handprothese verwendet ein halbautonomes Greifsystem, das das Objekt identifiziert, und nur durch eine einfache Menge an Benutzerfeedback findet das Greifen statt. Die Handprothese des KIT verwendet ein eingebettetes System, das durch begrenzte Ressourcen eingeschränkt ist. Es sollte jedoch eine Klassifizierung / Segmentierung und Erfassung von Objekten in Echtzeit erfolgen.

Diese Arbeit konzentriert sich auf die Objektsegmentierung. Eine kleine Studie über verschiedene Arten von Optimierungsmethoden für neuronale Netze und einige der neuesten neuronalen Netze für das eingebettete System wurde vorgestellt. Ein evolutionärer Algorithmus wurde implementiert, um eine neuronale Netzwerkarchitektur für verschiedene Arten eingebetteter Hardware zu finden, die hinsichtlich Geschwindigkeit und Segmentierungsgenauigkeit optimiert sind. Die Auswirkung der Tiefe auf die optimierte Architektur wurde ebenfalls abgeschlossen.

Das optimierte neuronale Netzwerk zum Zweck der Objektsegmentierung wurde für drei verschiedene eingebettete Hardware entwickelt. Für 13 Objekte wurde eine Segmentierungsgenauigkeit von 0.834, 0.843 und 0.939 für Mikrocontroller, FPGA bzw. Jetson TX1 erreicht.

Abstract

Prosthetic hands are a blessing for amputees. The main obstacle when they use prosthetic hand is the natural grasping of the object. The conventional myoelectric hand needs too much focus at the time of grasping any object. To reduce this cognitive load at the time of grasping is the current aim of the research. The KIT prosthetic hand uses a semiautonomous grasping system, which identifies the object, and just by a simple amount of user feedback, grasping takes place. KIT prosthetic hand uses an embedded system which is constrained by limited resources. But real-time object classification/segmentation and grasping should take place.

This thesis focus on object segmentation. A small study of different types of optimization methods for neural network and some of the state of the art neural network for the embedded system have been presented. An evolutionary algorithm has been implemented to find neural network architecture for different types of embedded hardware, which are optimized for speed and segmentation accuracy. Effect of depth on the optimized architecture has been also concluded.

Optimized neural network for the purpose of object segmentation has been designed for three different embedded hardware. For 13 objects, segmentation accuracy of 0.834, 0.843 and 0.939 for micro-controller, FPGA and Jetson TX1 respectively, has been achieved.

Table of Contents

Abstract	v
Zusammenfassung	vii
1 Introduction	2
1.1 Motivation	2
1.2 Problem statement	3
1.3 Outline	3
2 Robotic hardware platforms for network & inference	5
2.1 KIT Prosthetic hand	5
2.1.1 Semi-autonomous control	6
2.1.2 Current system	8
2.2 Embedded system	8
2.2.1 State of the art embedded systems	9
2.2.2 Software/Libraries for embedded inference	10
3 Fundamentals	12
3.1 Image segmentation algorithms	12
3.2 Neural network	14
3.2.1 Image classification and segmentation	15
3.2.2 State of the art neural networks for embedded systems	17
3.3 Optimization	22
3.3.1 Weight optimization	22
3.3.2 Network Architecture optimization	24
3.3.3 Hyper-parameter Optimization	26
3.4 Evolutionary algorithm	29
3.4.1 Parameters for EA	30

4	Conceptual design and data-set generation	33
4.1	Requirements	33
4.2	State of the art neural network architecture for semantic segmentation	34
4.2.1	DeconvNet (59)	34
4.2.2	SegNet (63)	35
4.2.3	ENet (64)	36
4.2.4	U-Net (66)	38
4.3	Neural network architecture	38
4.3.1	Parameter selection	39
4.4	Data-set generation	41
4.4.1	Pre-processing	44
4.4.2	Data-augmentation	45
4.5	Evolution	49
4.5.1	Implementation of the algorithm	49
5	Evaluation	52
5.1	Influence of augmentation	52
5.2	Results of Evolutionary Algorithm	53
5.2.1	Result for micro-controller	53
5.2.2	Result for FPGA	58
5.2.3	Results for NVIDIA Jetson TX1	62
5.2.4	Comparison	68
5.3	Impact of depth Information	69
6	Conclusion and Future Work	72
6.1	Conclusion	72
6.2	Future Work	73
	Bibliography	74

List of Figures

1.1	The KIT prosthetic hand (39)	3
2.1	Component integrated inside prosthetic palm (39)	6
2.2	Semi-autonomous control of KIT prosthetic hand	7
2.3	Comparison of number of operations for different embedded hardware	10
3.1	Biological Neuron (66)	15
3.2	A simple neural network	16
3.3	A basic Convolution Neural Network (CNN) (67)	17
3.4	Depthwise and Pointwise convolution in comparison to conventional convolution (23)	18
3.5	Factorized Hierarchical Search Space approach of MNasNet for network architecture (24)	19
3.6	Three main components of MNasNet framework (24)	20
3.7	Three different SqueezeNet architectures with simple bypass (middle) and complex bypass (right) (25)	21
3.8	Targeted Regularization by MorphNet (26)	22
3.9	Evaluation programming flow (11)	24
3.10	Particle Swarm Optimization algorithm flow (9)	25
3.11	Three steps of Neural Architecture Optimization (10)	26
3.12	Artificial Bee Colony optimization flow-chart (12)	27
3.13	Parameter selection for grid and random search (16)	28
3.14	Simplified EA procedure (6)	29
4.1	Architecture of DeconvNet (59)	35
4.2	Storage of Max-pooling indices in SegNet in comparison to FCN, where whole feature map is stored (63)	36

4.3	(a) shows initial block with side by side Max-pooling and convolution followed by concatenate layer, (b) ENet bottleneck module (65)	37
4.4	U-Net architecture (66)	38
4.5	Simple conceptual design	39
4.6	DEMO 13 data-set	43
4.7	From original image (left), background is removed (middle) and new background is added (right)	44
4.8	From original image (left), background is removed (middle) and binary mask is generated (right)	45
4.9	Same augmentation is applied to image and mask to avoid mismatch	48
4.10	Image processing, from capturing to creation of training and evaluation set	48
5.1	Effect of data augmentation on evaluation accuracy for micro-controller (gray) and FPGA (blue)	53
5.2	Conversion of accuracy, number of operation and memory usage in 10 generations for micro-controller	54
5.3	Visualization of the best architecture with filter and image size at each step (micro-controller)	57
5.4	Visualization of segmentation Ground truth (left), Predicted output(middle), Image (right) for micro-controller	58
5.5	Conversion of accuracy, number of operation and memory usage in 10 generations for FPGA	59
5.6	Visualization of the best architecture with filter and image size at each step (FPGA)	60
5.7	Visualization of segmentation Ground truth (left), Predicted output(middle), Image (right) for FPGA	63
5.8	Conversion of accuracy, number of operation and memory usage in 10 generations for NVIDIA Jetson TX1	64
5.9	Visualization of the best architecture with filter and image size at each step (Jetson TX1)	67
5.10	Visualization of segmentation Ground truth (left), Predicted output(middle), Image (right) for Jetson TX1	68
5.11	Network architecture with depth information	69

List of Tables

2.1	Currently implemented CNN on KIT prosthetic hand (41)	8
4.1	Description of DEMO 13 data-set	42
5.1	Best network for micro-controller after 10 generations	55
5.2	In detail description of the best network for Micro-controller	56
5.3	Best network for FPGA after 10 generations	61
5.4	In detail description of the best network for FPGA	62
5.5	Best network for Jetson TX1 after 10 generations	65
5.6	In detail description of the best network for Nvidia Jetson TX1	66
5.7	Comparison of three different hardware regarding resource constraint	69
5.8	Best network for Micro-controller with depth information after 10 generations .	70

Chapter 1

Introduction

Each year number of people become amputees because of unfortunate accidents. Prosthetic technology has made significant improvement in last decade. But still traditional myoelectric prosthetic hands are not very user-friendly. It needs constant user attention at the time of grasping any object, which, on one hand, makes multitasking a bit difficult and on the other hand, it makes user feel uncomfortable when around others.

Within the scope of the Intelligent Orthotic and Prosthetic (INOPRO) project intelligent control systems are being developed, which assist the user in grasping process by understanding the user environment. Recognizing the object is a must when it comes to choosing a grip for grasping task. For that, object detection algorithms, which helps in identifying object class and its pose, is necessary.

1.1 Motivation

Many research have been conducted to make this task autonomous. The KIT Prosthetic hand (see figure 1) has been developed to make the task of grasping semi-autonomous. It is a sensor-integrated, 3D printed hand with underactuated mechanism. It is equipped with embedded hardware (Cortex-M7) and a RGB camera in the base of the palm. It is underactuated with 2 motors and 10 degree of freedom.

The RGB camera helps in understanding the environment and thus the object class and pose. Once the object has been recognized, grasping is done by means of the motors. There is a display on the back of the hand, where the user can see the grasp recommendation. Which makes the task of grasping the object easier, reducing the cognitive load on user's mind.

Though understanding the environment and recognizing the object autonomously is a challenging task. It is a key element which is required to enable semi-autonomous process.



Figure 1.1: The KIT prosthetic hand (39)

1.2 Problem statement

Image captured by the RGB camera must be processed to get the information about objects. This is done through neural networks. In this thesis, captured images are used to train a neural network, which learns to detect objects in the environment.

Task of object detection must be done by on-board embedded hardware, which has many resource constraints. Thus neural network must be optimized as per hardware resources. Number of images for different classes was captured by the RGB camera and given as an input to different networks. Which were trained and then optimized using Evolutionary Algorithm (EA).

To design scalable network, which can be implemented on different types of hardware, network is optimized with a particular fitness function. That fitness function takes MAC operations and memory demand into account. Network is trained and optimized with this fitness function and the best network has been found.

1.3 Outline

In this thesis, chapter 2 explains about the KIT prosthetic hand in details followed by the currently employed object detection system. Additionally it mentions state of the art embedded hardware and software/library for interface.

Chapter 3 explains some fundamentals about neural network and some state of the art neural network for embedded systems. Some of the ways of optimizing neural network has been then mentioned and explained. At the end, parameters that are useful for optimizing neural network has been presented.

Chapter 4 presents some of the state of the art neural network for semantic segmentation followed by the initial design of the neural network. It is followed by data-set generation with

pre-processing and augmentation. At the end, evolutionary algorithm has been applied to the neural network for different hardware constraints and the results of which have been presented in chapter 5. Best network for all hardware has been presented and a small comparison of all the best network has been presented. Then depth information was also included in training to find its effects on the generated network.

Finally, chapter 6 concludes the thesis.

Chapter 2

Robotic hardware platforms for network & inference

In this chapter, a brief introduction on semi-autonomous control, that has been used in some known prosthetic hand, is given (section 2.1.1). In section 2.1.2, current system and performance of the neural network, that has been implemented on KIT hand, is shown. As, in this work, a neural network for different embedded hardware is to be designed, in section 2.2 some state of the art embedded hardware and some libraries for the interface have been mentioned.

2.1 KIT Prosthetic hand

Work in this thesis is mainly based on the KIT prosthetic hand (39). KIT prosthetic hand is a 3D printed hand which is based on TUTA/Karlsruhe mechanism (40). It was originally designed as a 20 DOF hand which can be controlled by only one actuator. KIT prosthetic hand has 10 DOF with two motors. The hand is so designed that the under-actuated mechanism allow fingers to wrap around an object of any shape. Four fingers are driven together via force-distributing transmission based in TUTA/Karlsruhe mechanism (40). One motor drives these four fingers and the thumb is controlled by the second motor. Its design resembles male hand and all the sensors, motors and embedded hardware is integrated in a palm like structure as shown in figure 2.1.

An embedded processor has been used to control the the prosthetic hand. It provides parallel camera interface and diverse peripheral serial interface and operates at 216 MHz. It is connected to four rechargeable batteries which can be charged through an integrated battery management system. It is also connected to an OLED color display which is located at the back of the hand. User can use it to know about battery percentage and other semi-autonomous functions. The

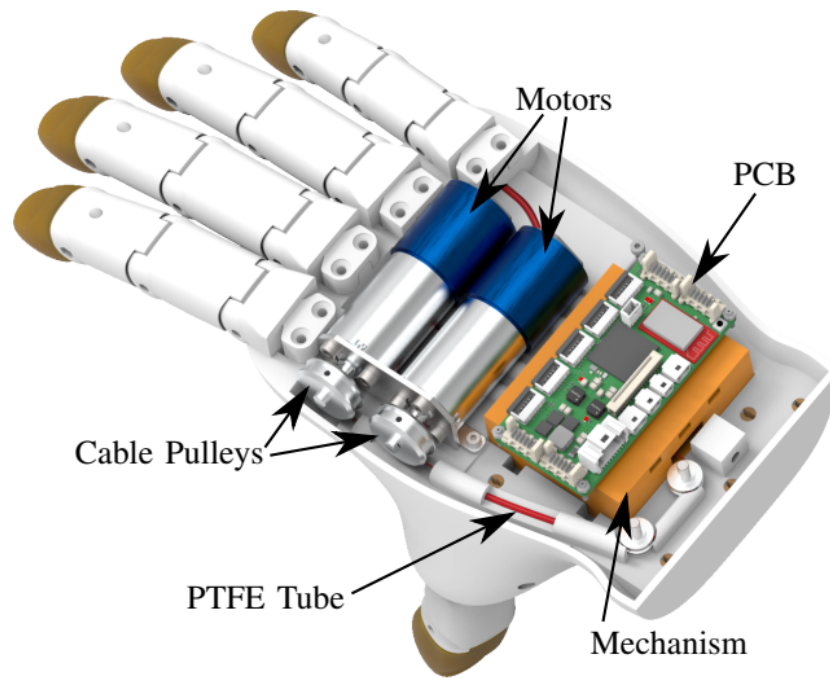


Figure 2.1: Component integrated inside prosthetic palm (39)

RGB camera at the base of the thumb is coupled with the OLED screen too.

2.1.1 Semi-autonomous control

In general, semi-autonomous control means, for the processes of grasping, very less user input is needed. It is basically implemented through these steps: First, the user adjusts the orientation of the hand towards the object it wants to grasp. Then with the help of specific classification algorithm, controller detects the object and selects an appropriate grasp type. At last step, the controller implements the grasp through closed-loop position/force control. The process for KIT prosthetic hand is shown in figure 2.2

Some research and studies have been done in field of object recognition with the help of neural network for prosthetic hand. These studies are mostly based on selecting best grasp as per the object. Selection of a grasp type according to the estimated object size was first suggested by Tomovic et al (44). A prosthetic hand was build which was based on myoelectric interface as shown in (45). The interface was used to choose from hand postures such as palm, pad and side position.

In last decade, these principals have been used to develop controller which can identify shape and size of objects from images and can select ideal grasp type for that object. As shown in (42), a dexterous hand (CyberHand) was developed in (2010) which is based on a vision system with

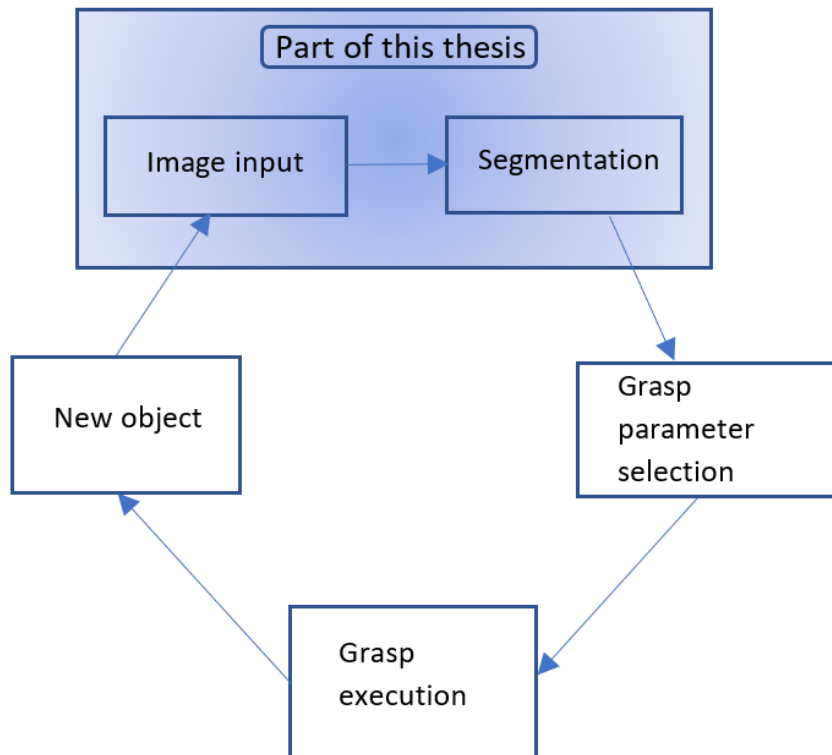


Figure 2.2: Semi-autonomous control of KIT prosthetic hand

rule-based reasoning. The controller was termed cognitive vision system (CSV) which resembles biological control. The overall task is divided between high-level controller and embedded controller. High-level controller detects the object, estimates the size to select an appropriate grasp and move the fingers accordingly. Embedded controller then does the grasping. As shown in (43), author carried out studies on a myoelectric hand and classified grasp types based on different images using a CNN.

In another approach (45), to get artificial proprioceptive feedback, augmented reality glasses with integrated camera was used. In order to get semi-autonomous and reactive control of a prosthetic hand, Marko Markovic (46) developed a context- and user-aware prosthetic controller integrating computer vision and inertial sensing.

In this thesis a neural network (NN) is developed for the task of image segmentation. As discussed in chapter 4, object segmentation is used to get pixel wise classification and thus the exact location of the detected object as an output. Some previous work on for this prosthetic hand is already done. A CNN for object classification, which is different than object segmentation, has been already developed.

2.1.2 Current system

Currently an ARM Cortex H7 microcontroller running at 400MHz has been used with an OmniVision 9650 RGB camera as hardware (41). A OLED display for visual feedback and two motors for controlling the finger has been implemented as well.

The current system is able to classify 13 different object from Yale-CMU-Berkeley (YCB) object set with an classification accuracy of 96..5% as stated in (41). The training was done using 100 images for each objects thus total of 1300 images. The network was optimized for size and speed to get real-time detection when implemented on hand. The results are shown in table 2.1.

Layer Type	Filter Shape	Output Shape	Ops	Runtime
Input	N.A.	$72 \times 72 \times 3$ (15.6 kB)	N.A.	N.A.
Convolution	$3 \times 3 \times 3 \times 4$ (0.1 kB)	$72 \times 72 \times 4$ (20.3 kB)	0.6 M	40 ms
MaxPooling	N.A.	$36 \times 36 \times 4$ (5.1 kB)	23.3 k	3 ms
Convolution	$3 \times 3 \times 4 \times 8$ (0.3 kB)	$36 \times 36 \times 8$ (10.2 kB)	0.4 M	13 ms
MaxPooling	N.A.	$18 \times 18 \times 8$ (2.5 kB)	11.7 k	1 ms
Convolution	$3 \times 3 \times 8 \times 16$ (1.1 kB)	$18 \times 18 \times 16$ (5.1 kB)	0.4 M	10 ms
MaxPooling	N.A.	$9 \times 9 \times 16$ (1.3 kB)	11.7 k	<1 ms
Convolution	$5 \times 5 \times 16 \times 32$ (12.8 kB)	$9 \times 9 \times 32$ (2.5 kB)	1.1 M	24 ms
AvgPooling	N.A.	$4 \times 4 \times 32$ (0.5 kB)	2.3 k	1 ms
Convolution	$5 \times 5 \times 32 \times 64$ (51.2 kB)	$4 \times 4 \times 64$ (1 kB)	0.8 M	23 ms
AvgPooling	N.A.	$2 \times 2 \times 64$ (0.3 kB)	1.2 k	<1 ms
Dense	$2 \times 2 \times 64 \times 13$ (3.3 kB)	13 (13 B)	1.2 k	<1 ms
Total	68.9 kB weights	48.8 kB activations	3.2 M	115 ms

Table 2.1: Currently implemented CNN on KIT prosthetic hand (41)

2.2 Embedded system

Embedded systems are microprocessor-based systems of hardware which are specifically designed to handle particular task. First embedded system was developed in 1960s by Dr. Charles Stark Draper for Apollo Guidance Computer. It was designed to provide mission-critical calculation by collecting data automatically. First commercially available microprocessor unit was designed by Intel in 1971, the Intel 4004 (47). In 1978, to improve the design of embedded system, the National Engineering Manufacturers Association released a standard for programmable micro-controllers.

In this thesis a method to design neural network for different types of embedded system has been discussed. As discussed in section 3.2, neural network requires more computational power and storage space. Recently many embedded hardware have been designed which can run specific

neural network efficiently. Some state of the art embedded hardware, which are suitable for neural network computation, are mentioned in the next section.

2.2.1 State of the art embedded systems

In this section information on some state of the art hardware, which are well optimized and capable of supporting high computation, are given. Some of the main features of those hardware have been mentioned and Comparison regarding operation per second has also been done.

- **Movidius**

Movidius are Intel's SoCs which are specifically designed for low power on-device computer vision and neural network application. Vision Processing Units (VPUs) are specially designed to provide acceleration for computer vision application and general purpose processing. There are two variant of Movidius hardware, namely, Myriad 2 and Myriad X. Intel's other hardware for low-power and optimum performance is cyclone FPGAs.

- **NVIDIA Jetson**

This is a high-performance embedded system-on-module from NVIDIA. They are very well suited for computer vision, deep learning and GPU computation for constrained environment. Some different variants are Jetson TX1, Jetson TX2, Jetson Nano, Jetson Xavier NX, Jetson AGX Xavier Series.

- **GreenWaves GAP8**

It is a RISC-V chip based system with hardware acceleration for convolution network and which is also optimized for low power consumption.

- **Gyr Falcon Technology Lightspeur**

Range of systems which are optimized for better performance at low power consumption and for high speed. These hardware are very efficient and capable when compared with others in terms of operation per seconds as can be seen from figure 2.3. Some different variants are Lightspeur@2801, Lightspeur@5801, Lightspeur@2802, Lightspeur@2803.

- **ARM microNPU**

ETHOS-U55 is a processor which is specifically designed to accelerate ML interface in area-constrained embedded devices.

In this thesis, when designing neural network, focus has been given to the number of operation and memory used. Memory support for Myriad 2, Myriad X, Jetson TX1 is 4GB while for Jetson TX2, TX2i, Xavier NX is 8 GB. Jetson AGX Xavier series supports 32GB of memory. Comparison of number of operations for some of these state of the art network has been shown in figure 2.3. For Jetson TX1, TX2 and TX2i, number of operations are in TFLOPS (Tera Floating Operations Per Second).

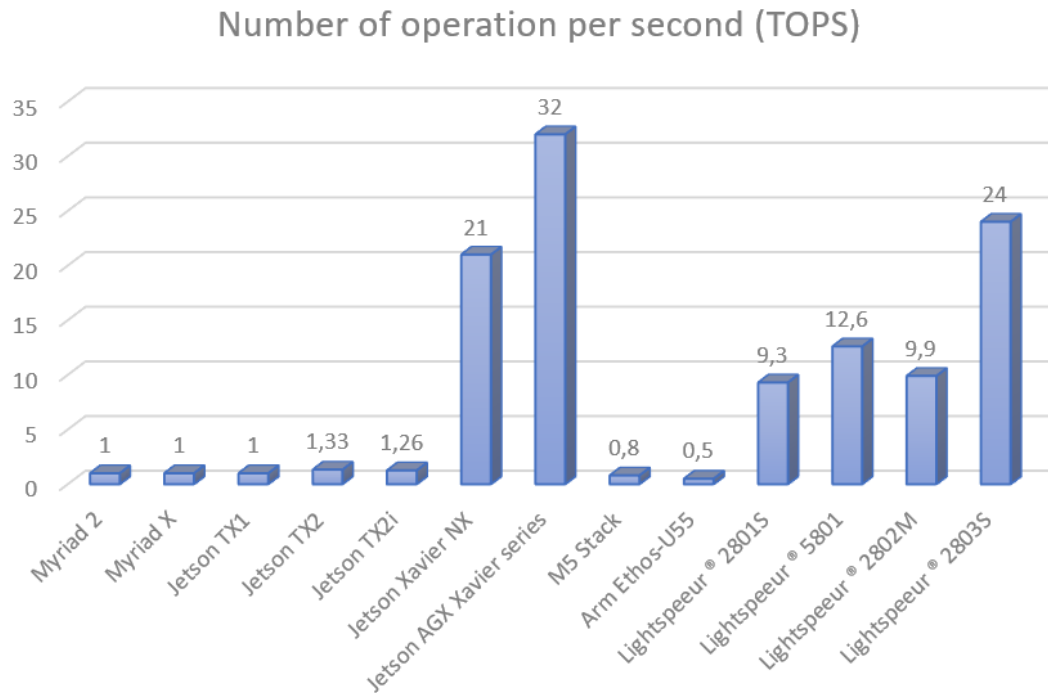


Figure 2.3: Comparison of number of operations for different embedded hardware

2.2.2 Software/Libraries for embedded inference

Embedded hardware has constraints when it comes to running programs. It needs particular software to run neural network, which is the main focus of this thesis. Models developed in specific programming language needs to be converted into embedded system friendly language or specific language or modules must be used to generate the models. Generally, codes are written in C and C++ language for embedded systems. But code from high-level languages like Python can also be converted to make them embedded system friendly.

- **Tensorflow Lite**

With the help of the Tensorflow Lite, model generated by Tensorflow (a python based module), can be converted into compressed model which can be implemented on embedded

systems. While converting, the model can be quantized which converts the 32-bit floats to more efficient 8-bit integer as mentioned here (48).

There is a special version of Tensorflow Lite for Microcontrollers too. It is designed to run machine learning models on microcontroller and similar devices with only kilobytes of memory.

- **uTensor**

It is a machine learning interface framework built on Mbed and Tensorflow which is extremely light-weight. Model which is constructed and trained in tensorflow is converted to .cpp and .hpp files. These files can be easily then implemented on embedded hardware (50).

- **Embedded Learning Library**

It is a Open Source library developed by Microsoft to allow user to design and deploy machine learning based models to resource constrained platforms and small single-board computers, such as Arduino and Raspberry Pi (49).

- **ARM Compute Library**

The Arm Compute Library contains function for convolution filters and Convolution Neural Network building blocks, like Convolution, Pooling, Activation, Soft-max etc. These functions are optimized for image processing and computer vision application (51).

- **CMSIS NN**

It is a collection of neural network which are efficient and specially developed to increase the performance and minimize the memory footprint of neural network on Cortex-M processor cores (52). The library features function for different categories, such as Convolution functions, Fully-connected Layer function, Pooling function etc.

- **deepC**

DeepC is a compiler and interface framework which is developed to make implementation of deep learning neural network on different embedded devices like FPGAs, micro-controllers, raspberry-pi, mobile phones and arm laptops. The main feature of deepC is to represent and optimize the common deep learning networks in high level graph IR and to convert the graph to minimize the memory utilization and optimize data layout for different hardware backends (53).

Chapter 3

Fundamentals

This chapter provides information on image segmentation algorithm, neural network and optimization methods. To design a neural network for embedded system, some of the state of the art neural network specially designed for embedded hardware is explained in section 3.2.2. Different optimization for neural network has been also explained in section 3.3 and at the end the selected optimization method, that is Evolutionary Algorithm (EA) and parameters, which can be optimized to get efficient neural network, have been described in section 3.4.

3.1 Image segmentation algorithms

Image segmentation is basically a method to segment an image into multiple segment. It is a pixel-wise segmentation which assigns a label to each pixel with a goal to identify exact location of an object in the image. In medical imaging, when applied to stock of images, semantic segmentation can be used to create a 3D reconstruction of a body organ with the help of interpolation algorithms. In machine vision, it can be used for autonomous car to detect different objects on the road and in surrounding environment.

As mentioned in (54), there are two main principles on which image segmentation is based: characteristic of a pixel and the information in nearby regions. Segmentation divides image into different nonempty, continuous and discontinuous subsets (55).

Image segmentation can be divided into main four categories. Edge-based segmentation, region-based segmentation, histogram-based segmentation, cluster-based segmentation and special theory-based segmentation.

- **Edge-based segmentation**

Edge can be considered as pixels which are at the end of one region, where intense discontinuity such as texture variety, gray value and color change takes place. This discontinuity or change can help detect a region boundary and thus exact location or size of a region. When it comes to accurate segmentation, noise plays a big role. Noise may cause fake edges. Gradient-based method is commonly use to detect edge.

Gradient-based method Gradient is the first derivative of the image $f(x,y)$. It detects the change in color value near the edge. If the change at the edge is noticeable and there is less noise, this method works well. Laplace of gradient (LoG) and Canny technique are some of the most used gradient-based method.

Filter-based method Here filter is defined as a matrix of certain shape with different weight values. The filter is then moved on the whole image as per stride chosen and element-wise multiplication is done. The value of weight in filter defines the output. Sobel filter and laplace operator are two commonly used filters. Sobel filter has two different filters for vertical and horizontal edge detection while laplace operator has single filter to detect vertical and horizontal edges.

- **Histogram-based method**

Histogram-based method are relatively less computation heavy in comparison to other methods because it passes through pixels only once. A histogram with the pixel value of the image is generated and region is generated thorough the peaks and valleys in the histogram. Pixel values can be gray value or color value. Because of the efficiency of this method, it is easier to apply this method to multiple frame. Though it is difficult to choose high and low value from histogram for single frame, for multiple frame, it is relatively easier.

- **Region-based segmentation**

Unlike Edge-based segmentation, region-based segmentation used a specific criteria to segment image into different regions. As mentioned in (56) some of the methods are thresholding, region splitting, region growing and region merging.

Thresholding It basically means to recognize a part of an image as a region based on a threshold value (T). Threshold value can be applied to whole image together (global thresholding) or it can be applied to a part of the image (local thresholding). Global thresholding divides a image into foreground and background based on the value of the pixel and threshold vlaue (T). As mentioned in (57), some of the global thresholding technique includes histogram concave analysis, iterative thresholding and entropy-based thresholding.

Region growing, splitting/merging Region growing is a method to define a region based on the property of neighboring pixels. Some pixels are selected at start and properties of neighboring pixels are evaluated based on some predefined criteria. If the properties of

pixels matches the criteria, it is defined as one region. This steps are then repeated until stopping criteria is reached. On the other hand, for region splitting and merging, some unconnected regions are selected and then based on selected criteria, merging or splitting of those areas takes place. It usually applied with theory based on quadtree data, as mentioned in (58).

- **Cluster-based/clustering segmentation**

Clustering is a method to divide data points into a number of clusters, such that data points in same cluster have similar values. Mainly used clustering method is k-means. First, some clusters (k) are randomly generated. To these clusters, data points are randomly assigned. After calculating center of these clusters, for each clusters, distance of data points from the center is calculated. Depending on this distance data points are reassigned to different clusters. Center of the newly formed clusters are then calculated again and data points are again reassigned. This procedure is repeated until predefined number of iteration or there are no change in the clusters. Advantage of k-mean algorithm is that it is very simple and easy to understand.

- **Special theory-based segmentation**

As mentioned in (54), there are number of theory-based segmentation algorithm, such as fuzzy mathematics, wavelet transformation, morphology and artificial intelligence etc. Two of them are commonly implemented for practical use, namely, fuzzy-clustering segmentation and neural-network based segmentation.

In this thesis a neural network for image segmentation purpose has been designed. Basic information about neural network and some state of the art neural network has been mentioned in the section 3.2.

3.2 Neural network

Neural network is like a grid or a mesh of neurons, arranged in particular pattern. A biological neural network is made of neurons which are connected with each other. One neuron can be connected to many other neurons and thus they form a long chain of neurons from which information can be conveyed. The connection between them is called synapses. They are normally formed from axons to dendrites. Information from them can be conveyed through electronic signals, as shown in figure 3.1.

Biological neural network was first proved and theory of it was proposed by Alexander Bain (20) in 1873. When our brain gives command to any part of our body, signals are transformed through these neurons. Each time when signals are transferred, the connection between neurons gets stronger. It shows that re-occurrence of signals forms a memory.

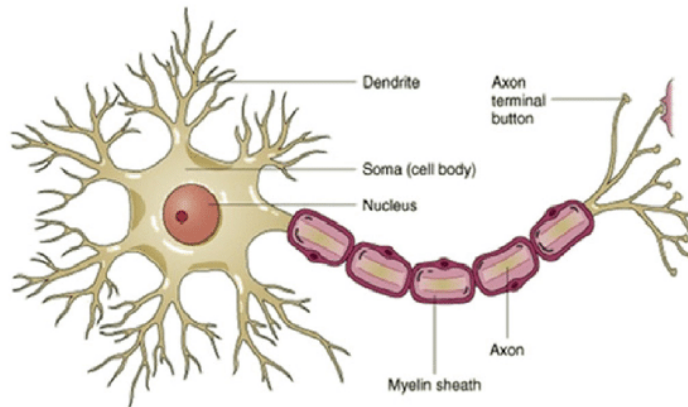


Figure 3.1: Biological Neuron (66)

In 1943, McCulloch and Pitts (21) use this information to create a model for neural network which was based on algorithms and mathematics. Many other neural network was created afterwards based on different algorithms. Though the research progress stagnated after 1969 for mainly two reasons. Computational power requirement and capacity of computers to run for too long. Computers were not capable of handling the calculation and time required by neural network at that time. In 1986, Rumelhart and MacClelland proposed a method to simulate neural process by using parallel distribution processing. Afterwards, neural network has been used in almost all fiends, to solve problems and get optimum results.

In general, neural network consists of some layers with number of nodes which are connected to each other. A simple neural network with one input, hidden and output layer is shown in figure 3.2.

It has been used in field of pattern recognition, thus, object detection and object segmentation tasks. Neural Network, in general, needs ample amount of input data for learning. In computer vision and image processing, neural network is used for tasks, like object classification, object tracking and segmentation.

3.2.1 Image classification and segmentation

Image classification

Nowadays, it is used for many tasks such as image annotation, face detection, activity recognition, tracking movement of objects. Before neural network was first used in object detection process, detection was carried out by approaches like, Viola-Jones object detection framework, Histogram of oriented gradients (HOG) and Scale-invariant feature transform (SIFT). But those approaches were too robust. They rely on features which were specified at the time of input. Neural network on the other hand, learn these kind of features itself.

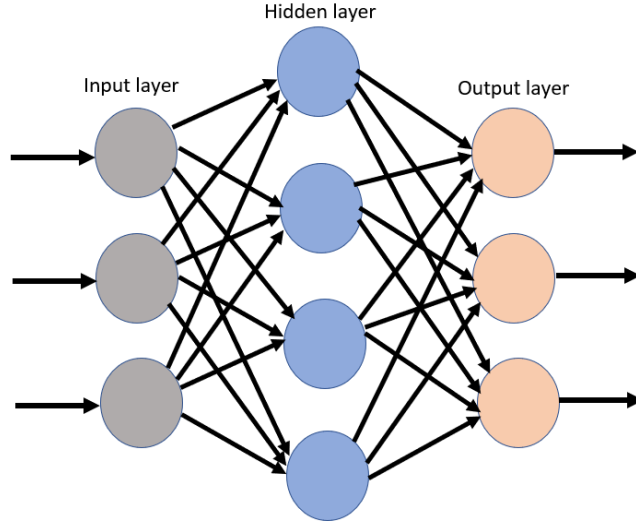


Figure 3.2: A simple neural network

A **Convolution Neural Network (CNN)** has proven to be very effective for image classification. Basic CNN is composed of input, output, and hidden layers. Hidden layer includes convolution, max-pool layers and fully connected layers, as shown in figure 3.3 Different object features are learned by the network with help of different convolution layers. Max-pooling layers, on the other hand, help to decrease the dimension of the image. At starting stage, it learn low-level features like dots, line, corner. At later stage with help of max-pooling layers, network learn high-level features.

In each layers there are number of nodes, which are called **Perceptrons**. Perceptrons of one layer is connected to Perceptrons of next layer. In mathematical term, this connection is defined as follows:

$$y_i = f(\sum(w_{i,j} * x_j) + b_i)$$

where, $f(\cdot)$ is a non-linear activation function,

$w_{i,j}$ is weight for i^{th} node of first layer and j^{th} node of second layer,

b_j is bias for j^{th} node,

x_i is the value of i^{th} node.

Convolution layer's parameters involves a set of learnable filters. Typically for a RGB image there are three channel. These filters extends through all three channels. One window of specific height and length is then passed through whole image and one 2-dimensional activation map is generated. *Pooling layer* helps reducing the dimension by clustering more neurons, which helps decreasing the complexity. This is done by either taking max value of the neuron from the clustered neurons, which is called max-pooling operation. It is also done by taking average of all clustered neuron, which is called average pooling operation.

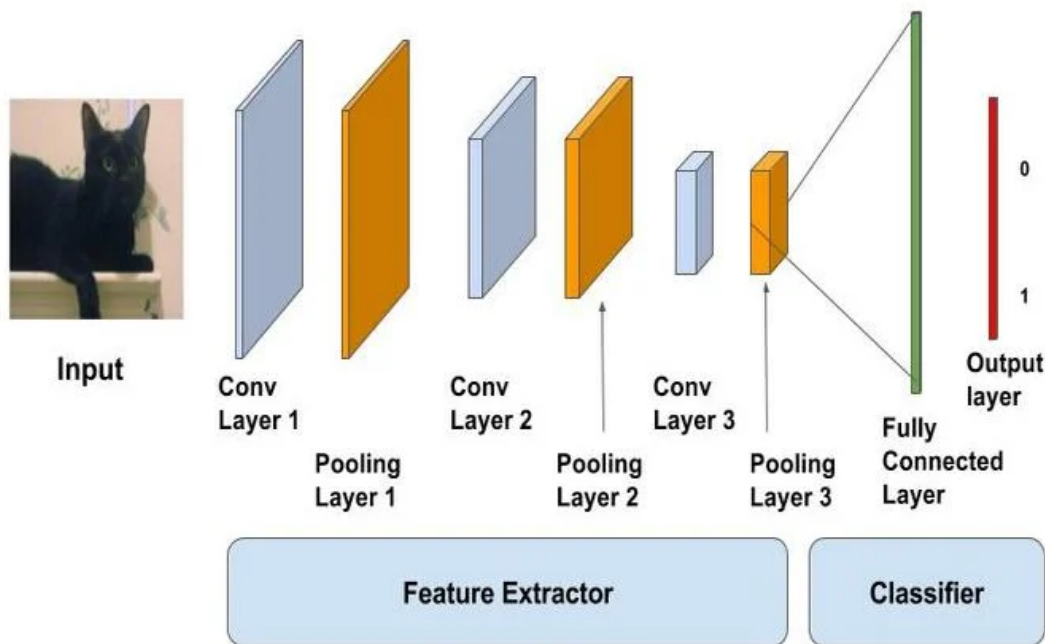


Figure 3.3: A basic Convolution Neural Network (CNN) (67)

Activation function are used to introduce non-linearity in the process which is a must when it comes to learning. Functions like Tanh, Sigmoid, ReLU (Rectified Linear Unit), Leaky ReLU and Softmax are used for different purposes. ReLU and Softmax are most used activation functions with excellent results.

Image segmentation

Unlike image classification, image segmentation is used for labeling each pixel of an image. Architecture of neural network which are used for image segmentation are different than that for image classification. As we need exact location of the object and not only the label of the object, they follow encoder-decoder type architecture. More details and some of different architecture are mentioned in section 4.2.

3.2.2 State of the art neural networks for embedded systems

As stated before, embedded system has constraints, such as Memory available, Computational power. But conventional neural network needs reasonable amount of computational power and memory.

In recent times, embedded systems are used everywhere, from our mobile to cars and trains. Nowadays, we use many small devices which accompany us everywhere which are very essential in many day to day tasks. These devices are portable and must be energy efficient. Many of those

devices need to stay as independent as possible.

When it comes to neural network for embedded systems, special types of neural network are needed in order to make embedded system use less power and memory. Power consumption and memory depends on number and types of layers, no of calculations. As shown in (22) not all of the layers use same memory and computational power. Convolution and Fully Connected (FC) layers use the most amount of computational power.

In recent times many research has been done for different types of neural network for embedded or resource constrained systems. Many different types of optimized and efficient neural networks has been designed.

MobileNet (23) is developed by google, where they have introduced an architecture with depth-wise separable convolution, which leads to a light weight network.

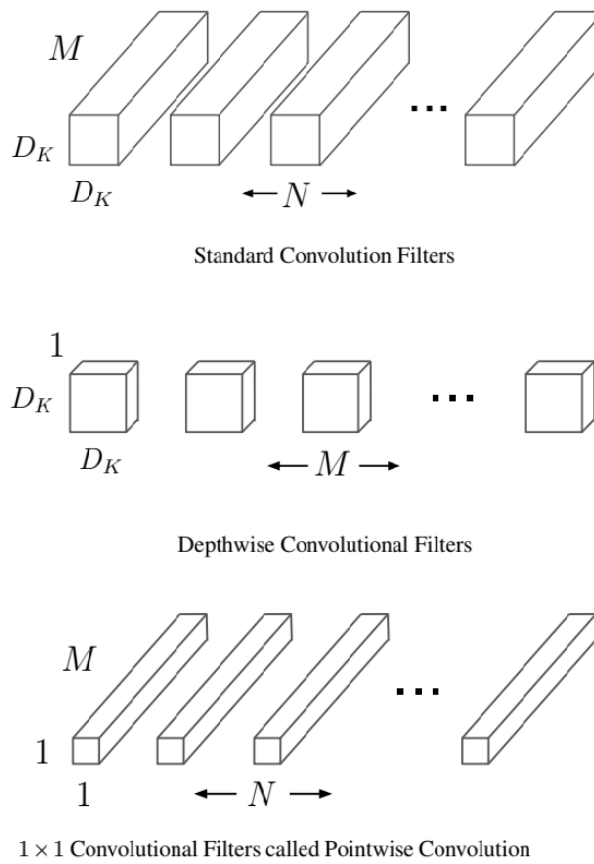


Figure 3.4: Depthwise and Pointwise convolution in comparison to conventional convolution (23)

Conventional convolution layer was replaced by a two convolutions. First one is depthwise convolution and the second one is a 1×1 convolution, which is called pointwise convolution. Depthwise convolution is applied to individual input channels which is followed by pointwise

convolution, which combines the output of the Depthwise convolution. In contrary to conventional convolution, depthwise convolution separates the task of filtering and combining, which leads to a decrease in computation and model size.

MobileNet uses 3x3 convolution filter which reduces the computation by 8 to 9 times when compared to conventional convolution operation. And as discussed, most of the computational power is used by convolution layers.

MNasNet (24) uses architecture search with reinforcement learning to find the model that achieves the best trade-off between accuracy and latency. A novel factorized hierarchical search space was proposed that encourages layer diversity throughout the network, which leads to balance between search space size and flexibility. Unlike search approach which optimize for indirect metrics, such as FLOPS, real-world inference latency was considered by directly running the models on mobile devices.

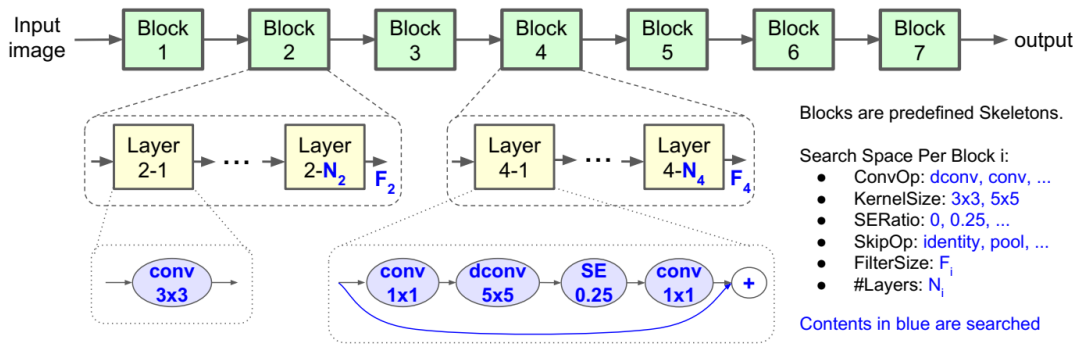


Figure 3.5: Factorized Hierarchical Search Space approach of MNasNet for network architecture (24)

A properly defined search space is critical for neural architecture search. Unlike some other approaches, where only few complex layers were searched and then those layers were repeated throughout the architecture, MNasNet focus on layer diversity, which leads to high latency and accuracy. In this approach, a factorized hierarchical search space has been introduced. It factorizes the neural network into distinctive blocks and then searches for the connection for each block separately. This approach leads to different layers in each blocks. Architecture of MNasNet is showed in figure 3.5.

MNasNet framework is divided into three main parts as shown in figure 3.6. At first, at each step, models are generated based on current parameters with the help of a recurrent neural network (RNN) based controller. Then those models are trained on target tasks to get the accuracy. At last, those models are run on the mobile devices to find the latency. After getting all those results, reward value is calculated based on a specific, application related equation. Then value of the parameters for next step is updated according to the reward value. These procedure is repeated

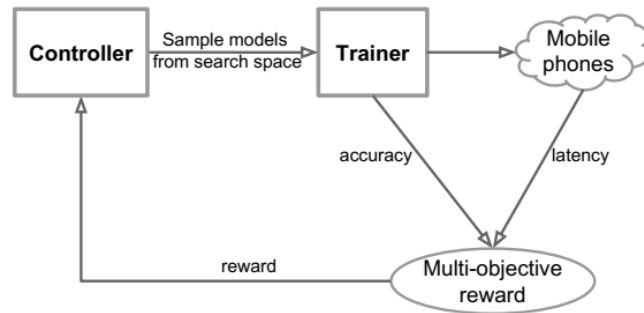


Figure 3.6: Three main components of MNasNet framework (24)

until it reaches to a maximum number of steps.

SqueezeNet (25) was released in 2016 and developed by researchers at Stanford University, DeepScale and University of California, Berkeley. It solely focus on compressing the model so that models can be used on resource constrained systems. It does so with keeping the accuracy of the model equivalent to the original model. It shows that fewer parameters has advantages, such as, realizable deployment of models on FPGA and embedded systems, efficient distributed training, less overhead when exporting models to clients.

Three main strategies were employed in this approach.

First one is to replace 3x3 filters with 1x1 filters. As we decrease the size of filter, low filter size directly affects the total parameter. 1x1 filter will use 9X less parameters than 3x3. Second one is to decrease the input channel to 3x3 filters. Third one is to downsample at the later part of the network. They have suggested that stride greater than 1 at the end part of the network increases the activation map, which leads to increase in accuracy.

Three types of SqueezeNet architecture is shown in figure 3.7. It starts with convolution layer which is followed by 8 Fire Modules which is followed by a convolution layer with high stride. Fire module includes: a squeeze convolution layer with 1x1 filters and a expand layer with mix of 1x1 and 3x3 convolution filters. Output of squeeze layer act as an input to the expand layer. Fire module has three hyper-parameters. Number of 1x1 filters in squeeze layer, number of 1x1 and 3x3 filters in expand layer. Those hyper-parameters are tuned to get better results. ReLU is used as an activation function and after fire module 9 a dropout layer was also included. The network is trained with variable learning rate with starting at 0.04 and decreasing gradually.

SqueezeNet showed some excellent results with getting AlexNet accuracy with only 5MB of parameters, while AlexNet has 240MB of parameters. They further showed a technique called Deep Compression, which compressed the parameter size to 500KB from 5MB.

MorphNet (26) was developed by researchers at google. It constructs smaller network from an existing network, which yields better performance. It produces smaller network by shrinking and expanding networks iteratively. At the time of shrinking it recognizes neurons which are

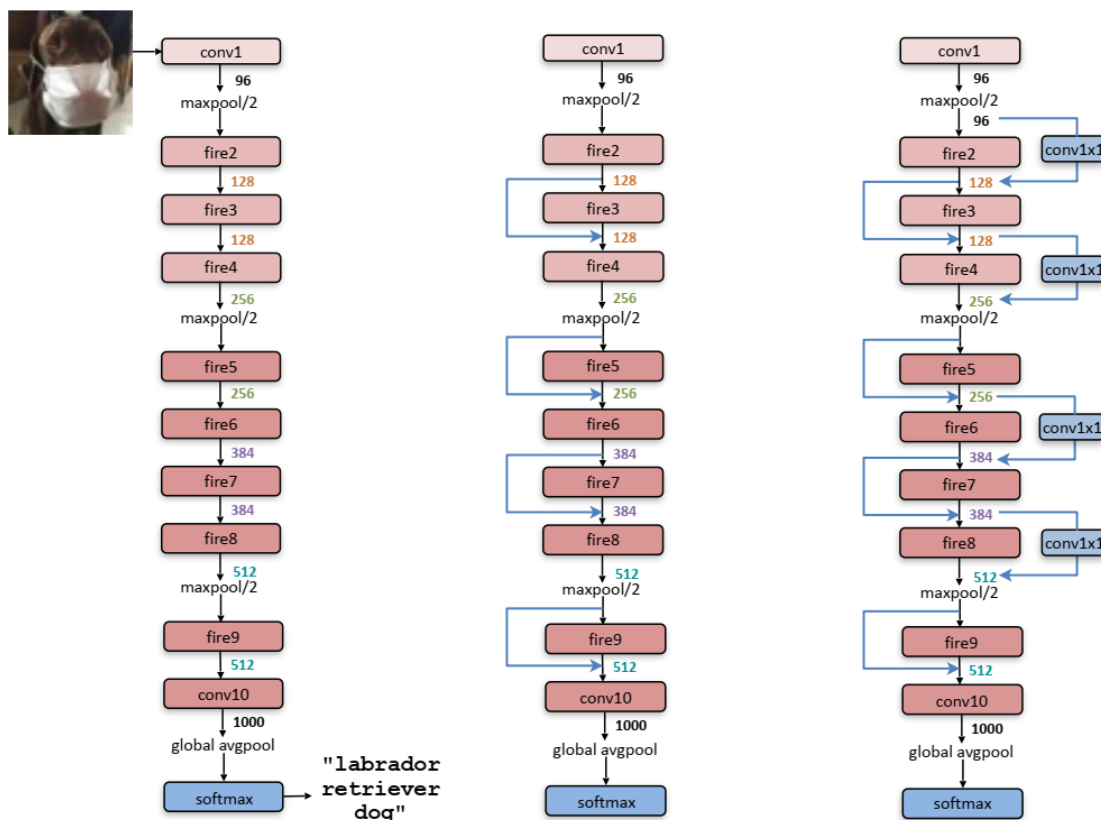


Figure 3.7: Three different SqueezeNet architectures with simple bypass (middle) and complex bypass (right) (25)

neither efficient nor necessary and removes them. It applies sparsifying regularizer such that cost of each neurons are included in loss function. It does so by taking resource into account.

At the time of expansion, it applies a width multiplier to each layer uniformly. If shrinkage and expansion are used together, if an inefficient layer with 50 neurons has 10 neurons after shrinking, expansion will increase that number to 11 only. But if an efficient layer has 40 neurons after shrinkage, expansion will increase neurons to 44. In general, MorphNet reallocates the resources to better places in network, which leads to increase in performance after using same or less resource.

As mentioned on their official blog (27), one of the benefits of using MorphNet is that it can be used for targeted regularization. It can be used to reduce number of floating point operations (FLOP) or size of the model by reducing weight as shown in figure 3.8.

Second thing is topology morphing. Morphnet is able to learn about the exact layer structure and number of neurons in it. If any layer has no efficient neurons, MorphNet removes the whole layer from the network. Other benefits are scalability and portability. It can learn about the whole

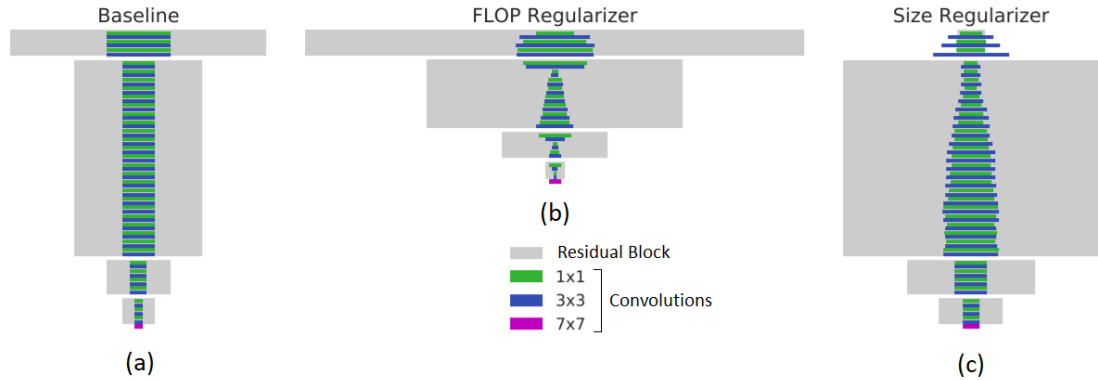


Figure 3.8: Targeted Regularization by MorphNet (26)

network structure in first step and can shrink or expand the network. It produces network which can be trained from scratch. It removes layers and weights but we don't have to take care of checkpoints, we can directly train the new network as we train any other network.

The code of this network has been open-sourced and can be found on git (28).

3.3 Optimization

Optimization of neural network is very essential to get better results, even without size and memory constraints. Success of neural network highly depends on the architecture, parameter used and optimization algorithm.

3.3.1 Weight optimization

The simplest optimization that takes place is training. Training is a kind of way to optimize the weight of layers and thus make network learn. This is done by gradient descent (GD) and back-propagation (BP) algorithm (1). This algorithm has some flaws, like weight values and surface shape. Neural network optimization is always a multi-dimensional problem, which causes gradient descent based algorithm to trapped at local minima. These kinds of problem can be certainly solved (2).

There are some well researched optimization algorithm to optimize weight of feed-forward neural network such as Differential Evolution, Evolutionary Programming algorithm, Particle Swarm Optimization algorithm and Genetic algorithm. Though these algorithm can be used for other kind of algorithms also, Differential Evolution has been used particularly for weight optimization, which has shown some good results too.

Differential Evolution (DE) is an algorithm that optimizes a problem in iteration. It improves a parameter in every iteration with respect to a given value. DE does not use gradient information

to optimize, which means it can be used to optimize a problem which can not be differentiated, which is a must in gradient descent. It was first introduced in (3). Comparison of this optimization method with other methods was shown in (4). Aspect such as robustness, efficiency, effectiveness and computational time of training was compared.

The way new population is generated is different for DE in comparison to other evolutionary algorithms. In first generation, a population with different weight is created.

$$\mathbf{P}_G = (\mathbf{W}_{1,G}, \dots, \mathbf{W}_{N,G}), G = 0, \dots, G_{max}$$

where,

$\mathbf{P}_G$ is population with N values,

$\mathbf{W}_{i,G}$ represents i^{th} population index and G is generation.

Main difference between DE and other algorithm is the way population for next generation is created. Vectors in current population, $\mathbf{P}_G$, are combined randomly to generate population for next generation.

$\mathbf{P}_{G+1} = \mathbf{U}_{i,G+1} = \mathbf{u}_{j,i,G+1}$ is created as

$$\begin{aligned} v_{j,i,G+1} &= w_{j,r3,G} + F * (e_{j,r1,G} - w_{j,r2,G}) \\ u_{j,i,G+1} &= v_{j,i,G+1}, \quad \text{if } \text{rand}_j[0, 1] \leq CR \\ &= w_{j,i,G}, \quad \text{otherwise.} \end{aligned}$$

where,

$i = 1, \dots, NP, \quad j = 1, \dots, D$ (D = total number of network weights)

$r_1, r_2, r_3 \in 1, \dots, NP$, randomly selected, ($r_1 \neq r_2 \neq r_3 \neq i$)

$CR \in [0, 1], \quad F \in (0, 1+]$.

Randomly selected parameters, thus, depends on the CR and F values. They can be chosen randomly and mutated as $v_{j,i,G+1}$ or $w_{j,i,G}$. This means that F and CR controls the rate of convergence of the process.

Evolutionary Programming (EP) Algorithm is very similar to Evolutionary Algorithm (EA). But as shown in (11), it is possible, that for some cases, EP can get better result than EA. EPNet has been described, which combines network architecture with weight learning. It differs from other algorithm based network in mainly two ways. First way is, that it maintains a healthy link between parent and its child, which is attained by techniques, like node splitting and partial training after each mutation. Generally, in mutation, learning of parents is destroyed by adding extra layers and connection to the architecture. Which leads to an unhealthy link between parents and child. Second, it attempts mutation sequentially. First deletion of node takes place and then new nodes are added. If deletion gets successful then new nodes are not added.

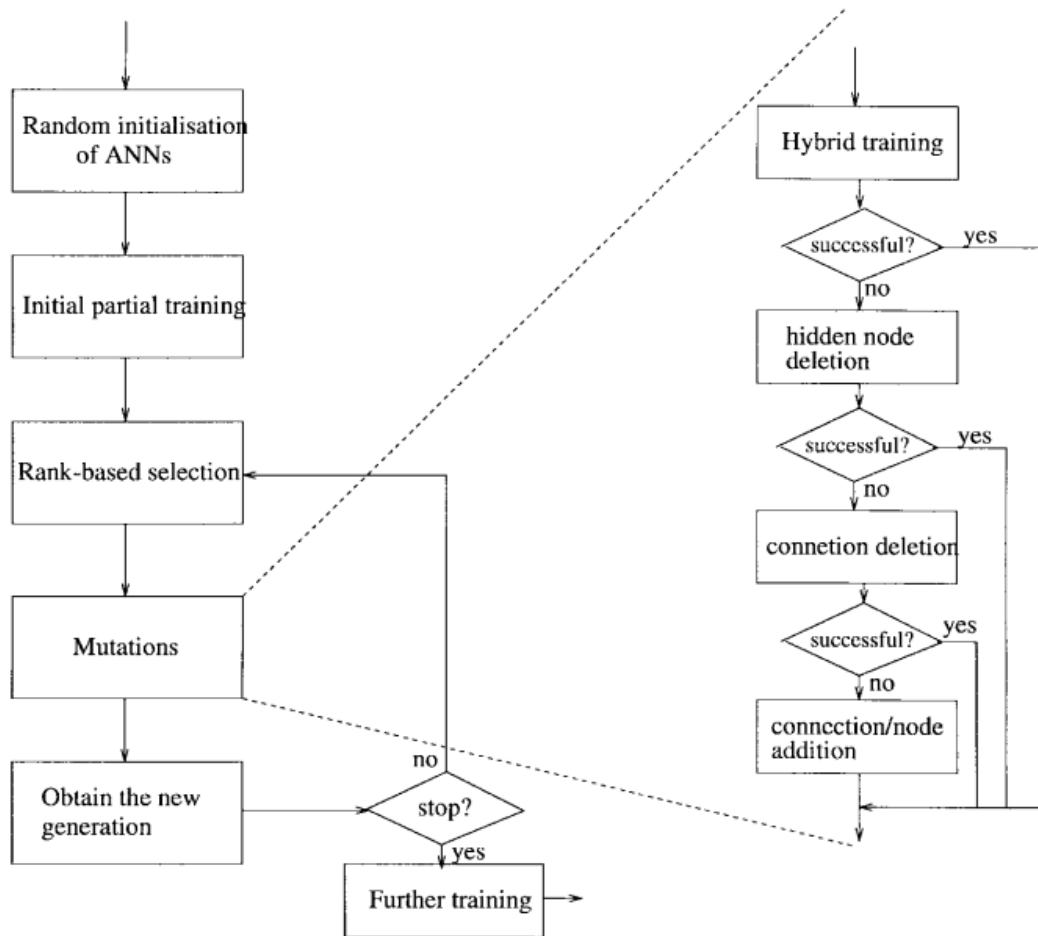


Figure 3.9: Evaluation programming flow (11)

3.3.2 Network Architecture optimization

Architecture optimization is an essential part when it comes to optimized neural network. Order of the layers can have significant effect on the learning of a network. There are mainly two algorithm which have been used to optimize neural network architecture. One of them is Particle Swarm Optimization (PSO) algorithm, which can also be used as weight optimization algorithm.

The Particle Swarm Optimization algorithm, in simple term, optimizes a problem by selecting random vectors (particles) in n-dimensional space and then trying to improve the solution by moving these particles in a given search-space as per a mathematical formula. Velocity of particles is affected by its best position. Particles are also steered towards optimum value in the search space, which is updated as optimum by other particles.

This method was first introduced by Eberhart and Kennedy in 1995 (7). Basic flow of this algorithm can be seen in the figure 3.10. Experiments conducted by authors in (5) involves

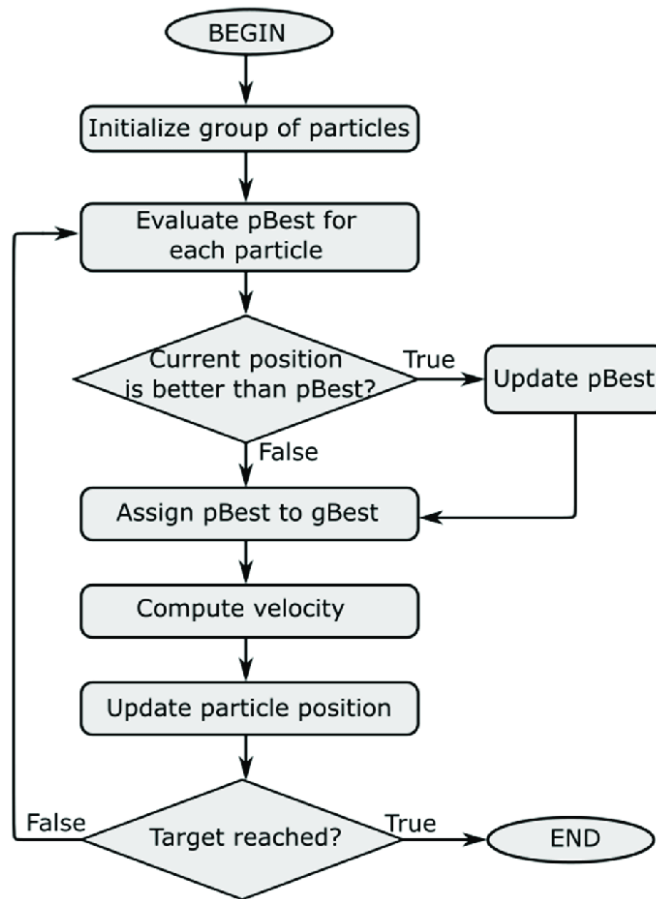


Figure 3.10: Particle Swarm Optimization algorithm flow (9)

classification and regression problems of neural network. Standard logistic activation function was used for classification problems. In (8) a different approach was taken, where input vector was divided into more sub-vectors. All sub-vectors was then optimized in their own search-space. Similar kind of approach was shown in (8), which they called Optimized Particle Swarm Optimization (OPSO). In this method, swarm was divided into sub-swarms and then optimized value of all sub-swarm was calculated.

Neural Architecture Optimization (NAO) is based on continuous optimization unlike reinforcement and evolutionary algorithms, in which optimization takes place in discrete space. This is relatively new approach in optimizing network architecture. It has main three key component as mentioned in (10). At start, network architecture is mapped on a continuous space by an encoder. Then a predictors predicts the accuracy by taking continuous representation of that network as input and at last, a decoder maps the network back to its architecture. Representation of which is shown in the figure 3.11.

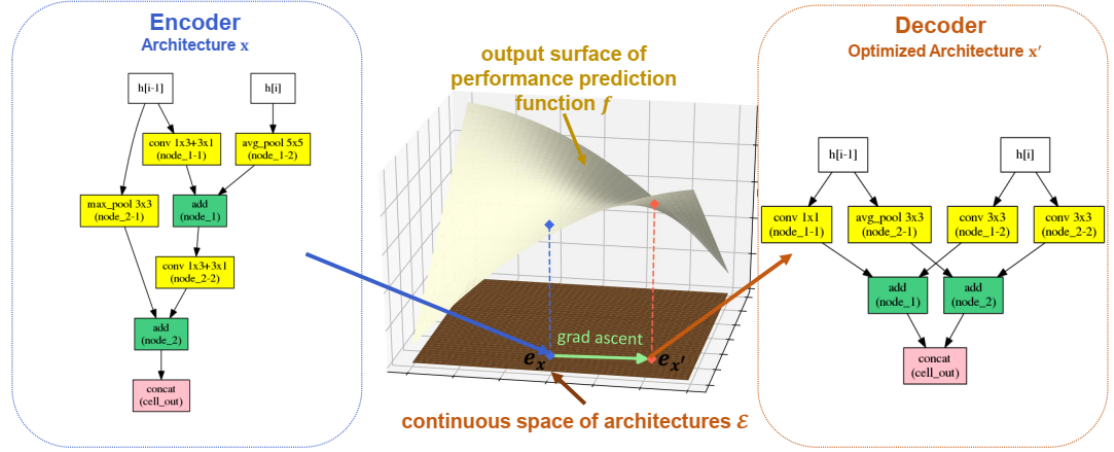


Figure 3.11: Three steps of Neural Architecture Optimization (10)

3.3.3 Hyper-parameter Optimization

Artificial Bee Colony (ABC) algorithm is an optimization algorithm which is based on honey bee swarm. It was first introduced by Karaboga in 2005. (13) Performance of ABC is compared with different algorithms like EA, DE and PSO (14).

In this algorithm, food source and amount of food is population and fitness function respectively. Number of onlooker bee is number of solution in the population. It involves scout bees, onlooker bees and employed bees. Each of them have their special task. As a first step, initial population P of N solution is generated. Each solution is a D -dimensional vector. Where D is equal to parameters.

In second step, population is evaluated in cycles. Employed bee searches for better food source and modify the location (solution) based on the quantity of nectar (fitness function). If the quantity (fitness function) is higher, it memories the new place (new solution). This information is then shared with onlooker bees, who evaluates and chooses food source according to the probability of quantity of nectar. If the probability is higher than the last location (solution), new position takes place of the old position. Which can be defined as follows:

$$p_i = \frac{fit_i}{\sum_{n=1}^N fit_n}$$

where fit_i is the fitness value of i^{th} solution. N is food source, which is employed bees.

New search location is defined by the following formula.

$$v_{ij} = x_{ij} + \phi(x_{ij} - x_{kj})$$

where $k \in 1, 2, \dots, N$ (number of population) and $j \in 1, 2, \dots, D$ (number of parameters). $i \neq k$. $\phi_{i,j}$ is a random number between $[1, -1]$.

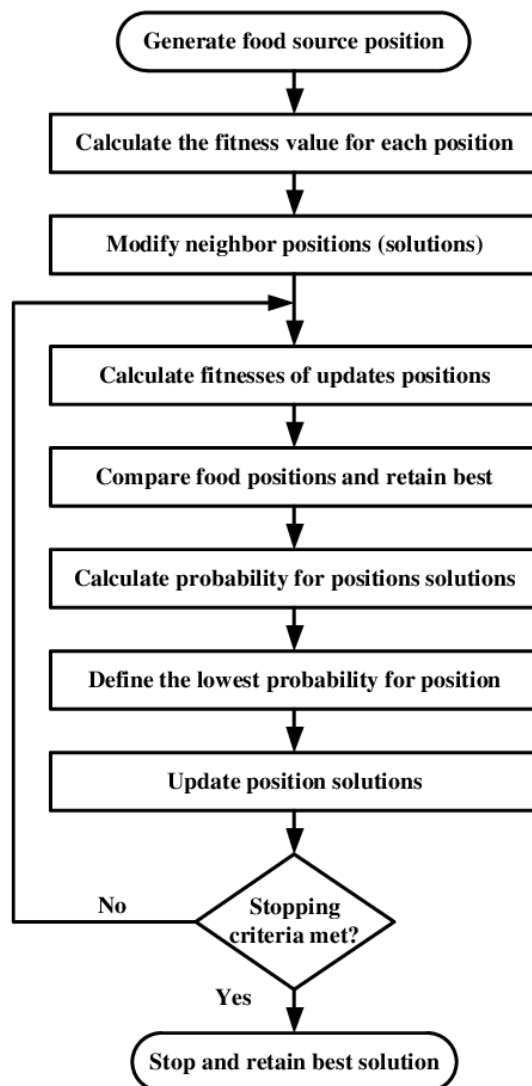


Figure 3.12: Artificial Bee Colony optimization flow-chart (12)

The discarded location (solution) is replaced by new location, which is done by scout bees. If the discarded solution is x_i , then it is replaced by new solution as formula defined below.

$$x_i^j = x_{min}^j + rand(0, 1)(x_{max}^j - x_{min}^j)$$

Performance of each new solution is compared with old one and is replaced only if it is better. Which is a greedy selection approach. This is done for a fixed amount of cycles which is important to decide in this algorithm. After fixed amount of cycles optimum solution is obtained. Flow

chart of this algorithm can be seen in figure 3.12.

Manual-, Grid-, Random search optimization are some of the most common strategies which have been used for hyper-parameter optimization. For Manual and Grid search methods, values of parameters are predefined and for combination of those values, model performance is calculated. As shown in (15), neural network has been used to find optimal surface roughness in a manufacturing process. Grid search optimization is used to optimize the hyper-parameter. For manufacturing, parameters; like feed, depth of cut and cutting speed, wear patterns, lubricant, workpiece deformation, vibration, chip formation, cutting parameters were used as parameters and values of which are defined with grid search method.

Random search, on the other hand, chooses values of parameters randomly. In another paper (16), Random search was used to optimize hyper-parameters. Random search, when used for neural network, finds better model in comparison to manual or grid search, by efficiently searching a larger configuration space. Random search is specifically better when used for less parameters. When less hyper parameter affects the last model, it is advantageous to use random search over grid and manual search methods.

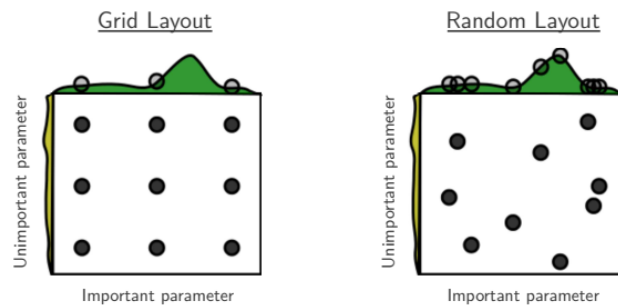


Figure 3.13: Parameter selection for grid and random search (16)

Bayesian optimization has been used for all kinds of optimization problems. It is particularly useful for noisy black-box function. It was first introduced by Jonas Mockus in 1974 (17). It treats function as an unknown and iteratively evaluates hyper-parameters configuration using prior probability distribution, based on current model, then updates them. It observes the function and tries to gather more information about the function. In turn it learns the location of optimum. As shown (18), (19), Bayesian optimization is used for optimizing hyper-parameters of neural network. It is shown that for general machine learning algorithms, optimum values for parameters were attained significantly faster.

Evolutionary Algorithm (EA) has performed very well for almost all kinds of neural networks. And thus has been selected as the main optimization method for this thesis. It can optimize hyper-parameter and also architecture of a neural network with good efficiency and less computational cost.

3.4 Evolutionary algorithm

Evolutionary Algorithm (EA) is a population based optimization algorithm. It is inspired by biological evolution. The main aspect of the EA is based on the concept of survival of the fittest. Where each sample in a population is optimized by crossover, mutation and reproduction. Crossover can be considered as parents producing children, where children inherit qualities from parents. Some children inherit good qualities from parents and some do not. When mutation takes place, children inherit qualities which are different from their parents. It works in an iterative manner. In each generation based on the fitness function best of the children are selected and taking them as parents, new children are created.

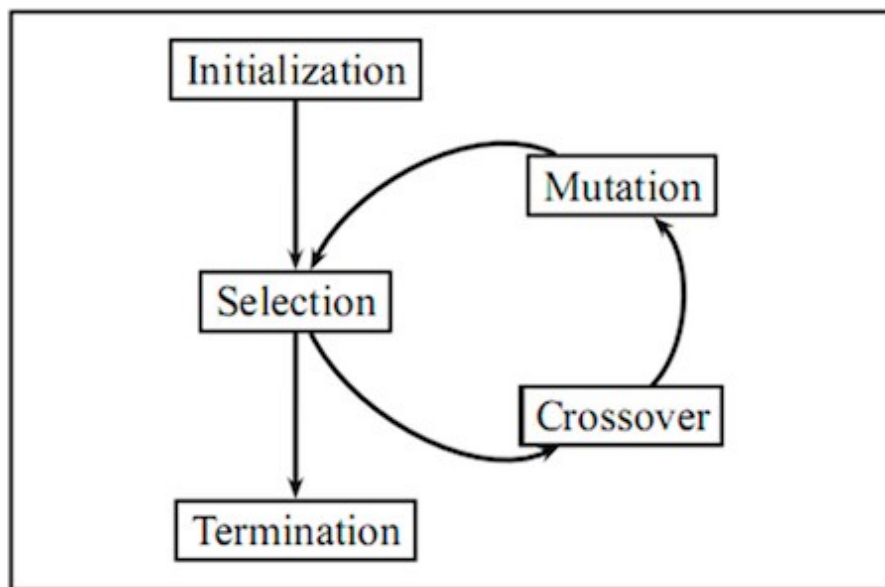


Figure 3.14: Simplified EA procedure (6)

In general, for neural network, at first generation number of population (networks) with selected parameters are chosen randomly. No of generation and no of network per generation is fixed. In each generation, networks are evaluated according to the fitness function. Then networks with best fitness function is selected and next population of networks is created by means of crossover, mutation.

For example, if 5 parameters with different values are selected to build the network, in crossover, the network for next population will get, for example, 3 parameter values from one parent and 2 parameter values from the second parent. If mutation takes place, generated network will have, for example, 1 or 2 parameter values which are different from both parents. In each generation, fixed number of children-networks are created from parent-network and then new networks are generated randomly so that we have same number of network each generation. Some networks are generated randomly so that we can have other parameters values which are not selected at the

first generation.

Evolutionary algorithm is not only useful for optimizing an neural network but as shown in (29), Evolutionary Algorithm (EA) was used to visualize an image which maximize the activation of a point of interest in neural network. Neural network learn to identify objects from an image but exact process behind it is not clear to us. They showed that from large image set, few images that maximize the feature's activation are selected. Then going back from this activated feature (the other ones being silenced) provides a visualisation of the activated feature and hence an idea of what made it react. Evolutionary Algorithm (EA) can work in the image space and evolve images at each generation which increases the feature's activation.

3.4.1 Parameters for EA

As mentioned above, EA optimizes a neural network based on specific parameters and fitness function. Fitness function is essential in process of optimization, but parameters are equally valuable. Selection of parameters plays a key role in getting accurate evaluation results.

- **No. of hidden layers**

Selection of hidden layers for a specific application is one of the difficult tasks when it comes to neural network generation. As mentioned in (30), too less layers can leads to high training error and high generalization error due to under-fitting. Too high layers can decrease the training error but still, due to over-fitting, high generalization error occurs.

- **Kernel size**

In a convolution layer a filter of a specific kernel size is used as a window to implement convolution operation. It is used to get local information from an image. Too small kernel size can provide too much information, which can cause confusion and too large kernel size can miss some information. It also helps in reducing the weight by reducing the connection between two layers, when used with stride greater than 1. Less connection leads to less floating point operation. Floating point operation is a key criteria for embedded systems. In general, kernel of 3x3 or 5x5 is used for neural network.

- **Feature maps**

Feature maps can be interpreted as different filters. As mentioned above, in a convolution layer, a window of kernel size is used over whole image to get local information. For an image windows with different pixel values can be selected. Different windows help in

identifying different features from an image. Filter with different pixel value generates different output images, which are called feature maps. Feature maps helps in learning more feature from an image but it also increases the size of the model and also number of floating point operations.

- **Learning rate**

Neural network are trained using stochastic gradient descent (SGD) algorithm or a variant of it. SGD estimates error gradient and updates the weight of the models through backpropagation. The amount by which weights are updated is called learning rate. It defines the speed at which the model learns. High learning rate can leads to oscillation in performance over epochs. Low learning rate can leads to a local optimum. In practice, it is good to take 0.1 or 0.01 as starting learning rate. Keeping learning rate same throughout the training is also not advisable. Changing learning rate overtime is called learning rate decay. It is good to take high starting learning rate and decrease it with epochs.

- **Dropout Layer**

As we train network for more epochs, many times networks tends to overfit. It shows impressive results at the time of training but shows poor performance when the model is evaluated on test data. This happens because the model learns the noise in the training data or of the training data is biased. During training, some nodes of layers are ignored. This is similar as if a layer has different node count and connection to preceding layer. Dropping out a node means removing that node from the network temporarily and also the incoming and outgoing connection of those nodes. It changes the topology of the network temporarily which helps other nodes learns about different features of images. That, in turn, leads to less over-fitting and thus less evaluation error. The nodes to be dropped out is randomly selected. It has effect of thinning the network size during training, so it is preferable to use dropout layer while training wider network.

- **Skip/ Residual connection**

Skip connection has become a norm for deep networks. As mentioned in (31), The difficulty in training deep neural network is due to the singularity caused by the non-identifiability of the model. These singularity slow downs the learning of networks and in some case it degrades the learning. It is shown that skip connection can eliminate singularities which can be caused by permutation symmetry of nodes and linear dependence of nodes. Since introduction of skip connection, it has shown excellent results for improving accuracy of deep neural networks (32), (33). Skip connection can also be used as a means of transferring the location information from one layer to another, which is very helpful for

semantic segmentation. As in this work, semantic segmentation is a focus of work, skip connection has been used as a parameter inspite of network not being a deep neural network.

- **Batch size**

Batch size is the number of training images that are used at the time of training to update the network parameters. As shown in (34), high batch size increase the recognition accuracy of a neural network. Normally, batch size is set to a power of 2, which was shown in the (35) by the effective work of optimized matrix operation libraries. In some cases,(36), (37) it is also pointed out that batch size equal to multiple of 10 has shown better performance. It was proved (38) that, high batch increase the recognition accuracy. On the other hand, high batch size leads to elevated computational costs and thus high training time.

Chapter 4

Conceptual design and data-set generation

In this chapter, design of the neural network, as per available resources and requirements, has been presented (section 4.1). Before creating the final conceptual design, some of the state of the art neural network have been studied (section 4.2) and some of the features from those networks have been used to finalize the design. Dataset of available object is also generated with the help of available RGB camera of the prosthetic hand, as shown in section 4.4. That data-set is then pre-processed to simulate real word scenario by changing the background and more images are generated using data-augmentation function of keras.

Two things, those must be kept in mind while applying optimization method and creating neural network, are embedded hardware and the purpose for which the neural network is to be designed, that is, semantic segmentation. As it has been mentioned in chapter 3, embedded systems are resource constrained hardware and thus some special requirements must be met while creating neural network.

First thing that we must take into consideration is the requirements those should be met while designing the neural network.

4.1 Requirements

Some of the requirements for the embedded hardware (ARM Cortex M7 microprocessor) currently used in KIT prosthetic hand are as follows:

- **Computational Power**

It is limited by processor's clock frequency which is 400MHz. All the processes, such as motor, display, IMU also uses the power. While running the neural network, these

components' working must not be delayed. Designed network must be power efficient.

- **Memory**

Currently available memory on Cortex M7 microprocessor is 1 MB of RAM and 2 MB of storage. While processing images, neural network uses RAM at each stage depending on the dimension of the image and type of layer.

- **Segmentation time**

The most important requirement is that neural network should not take too long to do the segmentation task. As after image processing, hand has to do the task of selecting a specific grip and also execution of selected grip. It should not take more than 150ms to do the image processing.

- **No of objects**

Designed network must be able to differentiate between a defined number of objects. In this thesis a network for 13 different object has been designed. Objects types and numbers are mentioned in section 4.4.

The other criteria, that this designed neural network must meet, is accuracy. Even though the network must fulfill these requirements, one main criteria is good segmentation accuracy. Without good results, it doesn't matter even if the network meet all these requirements. In short, the designed neural network must be so designed that it finds a solution which fulfill these requirements with good, if not excellent, accuracy.

To find which parameters are important when it comes to semantic segmentation, research on some of the already optimized and well known network for semantic segmentation was done. Some of the state of the art network are explained in the next section.

4.2 State of the art neural network architecture for semantic segmentation

In recent times, due to the increased interests in home-automation devices, augmented reality wearables and self-driving vehicles, need for semantic segmentation algorithms that can run on small devices has been increased. CNN has been very successful in classification, detection (60), (61) and action recognition tasks. CNN have also been used for semantic segmentation, but CNN provides coarse spatial results.

Several neural network therefore have been designed for segmentation task. In this section, some of those architectures have been mentioned and explained.

4.2.1 DeconvNet (59)

As mentioned in the paper (59), Fully convolutional network (FCN) has two main limitations when it comes to semantic segmentation. Due to the fixed-size receptive field, it has problem

segmenting too large or too small objects. And details of an object is often lost or smoothed because the label map is too coarse. After taking these two limitations into consideration, DeconvNet architecture has been designed. It is divided into two parts, convolution network which transforms the image into multidimensional feature maps and deconvolution network which does object segmentation from those extracted features.

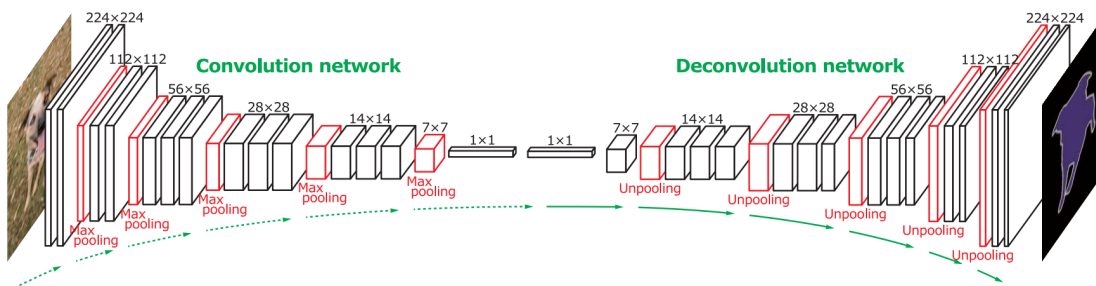


Figure 4.1: Architecture of DeconvNet (59)

Architecture of DeconNet, shown in figure 4.1, is based on VGG 16-layer net (62). It consists of total 13 convolution layers and 2 fully connected layers are augmented at the end for class-specific projection. Deconvolution part is a mirror image of the convolution part.

In deconvolution part, unpooling layer is used which increase the dimension according to the filter size used. At the time of max-pooling operation, location of maximum activation is recorded, which are used at the time of unpooling to place those value to their original pooled location. Deconvolution layer differs in a way it associate a single input activation with multiple output. Lower level layers captures overall shape of an object while higher level layers captures detail about class of the object in image. This helps network in getting directly class specific information into account for semantic segmentation.

To optimize the network further, batch normalization at the end of all convolution and deconvolution layer is used. Also training is done in two steps. For the first step of the training, examples are created, such that objects are in the middle of the cropped image. It is used to help reducing the search space for semantic segmentation. In second stage, object proposals were used to construct more challenging examples which helps network robust to misalignment of proposals in testing.

4.2.2 SegNet (63)

SegNet (63) is a fully convolutional neural network. This network is also based on VGG16 (62), but it differs in a way the decoder upsamples the lower dimensional feature maps. At the time of maxpooling operation, the pooling layer stores the indices, which is then used at the time of unpooling. This eliminates the need to learn to upsample. Segnet is designed to be efficient in terms of memory and computational time during inference. It is also smaller in terms of training parameters when compared to other deep neural network and can be easily trained by stochastic

gradient algorithm.

As the architecture of the SegNet (63) is based on VGG16 (62), it resembles the architecture of DeconvNet (59) a lot. Each convolution layer in encoder part is followed by a batch normalization layer. It is followed by ReLU activation function and a 2x2 max-pool layer with stride equals to 2.

Max-pooling layer makes the location information inaccurate. To reduce that loss, it is necessary to capture and store that information before the sub-sampling takes place. This paper suggest an efficient way to store this information. The way is to store the indices of the maximum feature value in each pooling window for each encoder feature maps. It can be done using 2 bits for each 2x2 pooling window which is more efficient in comparison to storing whole feature maps in float precision. Upsampling in decoder network is done by using the memorized max-pooling indices from the corresponding encoder feature maps. It is shown in figure 4.2.

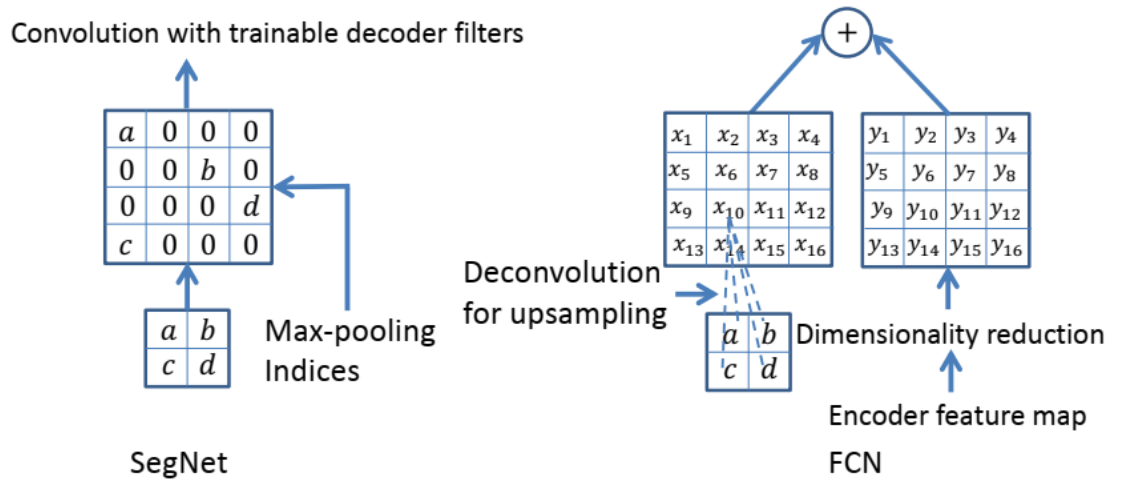


Figure 4.2: Storage of Max-pooling indices in SegNet in comparison to FCN, where whole feature map is stored (63)

To each feature maps, batch normalization is applied. For the last layer, softmax function is used to give input that has channel equals to the number of classes to segment. The output segmentation is predicted as a possibility of the pixel to belong to a particular class.

4.2.3 ENet (64)

ENet (Efficient neural network) (64) is, as its name suggest, a network specially created for tasks requiring low latency operation. Both of the above described network are based on VGG16 architecture, which is too large model for multi-class classification. Because of the execution time and computational power requirement they become very difficult to implement on battery-powered devices.

The unique architecture of ENet is divided into several stages. It also follows encoder-decoder approach but encoder is not mirrored in decoder. A part which resembles ResNets (65) structure, is also used in this architecture. As shown in figure 4.3, as an initial block of the architecture, Max-pooling and convolution is done side by side and then they both are concatenated to form a layer of 16 channel. It also has unique bottleneck module. Convolution operation in that module is either regular, full convolution or dilated.

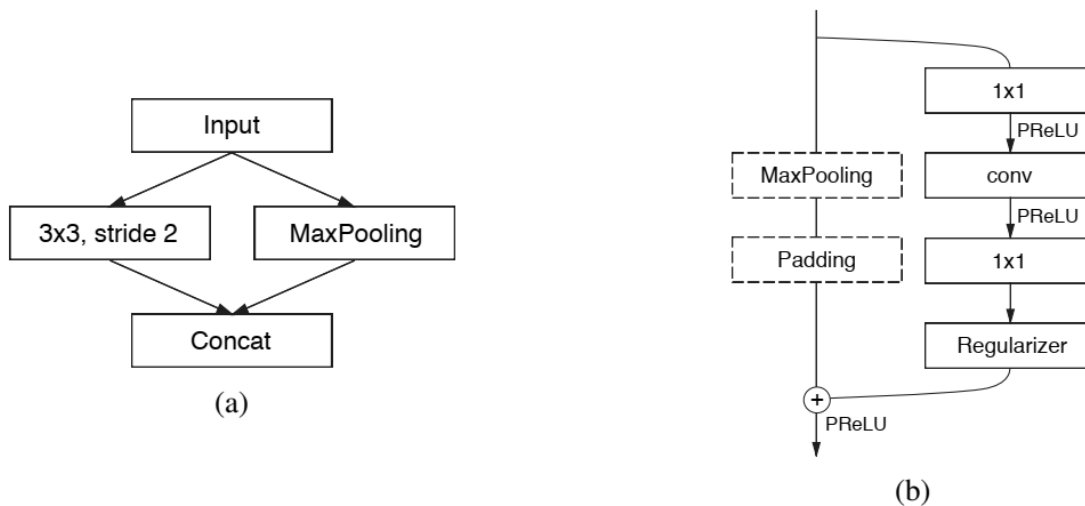


Figure 4.3: (a) shows initial block with side by side Max-pooling and convolution followed by concatenate layer, (b) ENet bottleneck module (65)

There are some design choices which makes this architecture efficient and light weight. First one is **feature map resolution**. It follows SegNet (63) for this and saves indices of max-pooling unlike saving whole feature map. Other one is **early downsampling**. It follows the approach that early layers should not contribute for classification task, and thus at early stage, only small set of feature map is used. Second approach it follows is that visual information is highly redundant and can be compressed into more efficient representation. That's why at the starting level two blocks heavily reduce the input size.

It also uses different **encoder size**. Unlike SegNet (63), in this network architecture, encoder is larger than decoder. It follows the idea that encoder should work on smaller resolution data and filtering, while decoder should only upsample the output of encoder and fine tune some details. **Factorizing filter** approach has also been used. It suggest that each $n \times n$ filter can be divided into two smaller ones following each other: one with a $n \times 1$ filter and the other with $1 \times n$ filter.

Other interesting feature that has been used is **dilated convolution**. Inside several bottleneck modules, regular convolution is replaced by dilated convolution. It gives wide receptive field, so it can perform classification by taking a wider context into account. It also requires less number of operation in comparison to regular convolution.

4.2.4 U-Net (66)

U-Net is a fully convolution network and it also follows encoder decoder approach but it uses skip connection which separates it from other state of the art network. Its architecture is shown in figure 4.4. Other thing which is different in this network is that in encoder part all convolution has 3x3 kernel size and all max-pool layer has 2x2 kernel size with stride 2 which decreases the dimension to half at each stage. Also the convolution layer is without padding which decrease the dimension, too. In decoder part each convolution has 2x2 filter size.

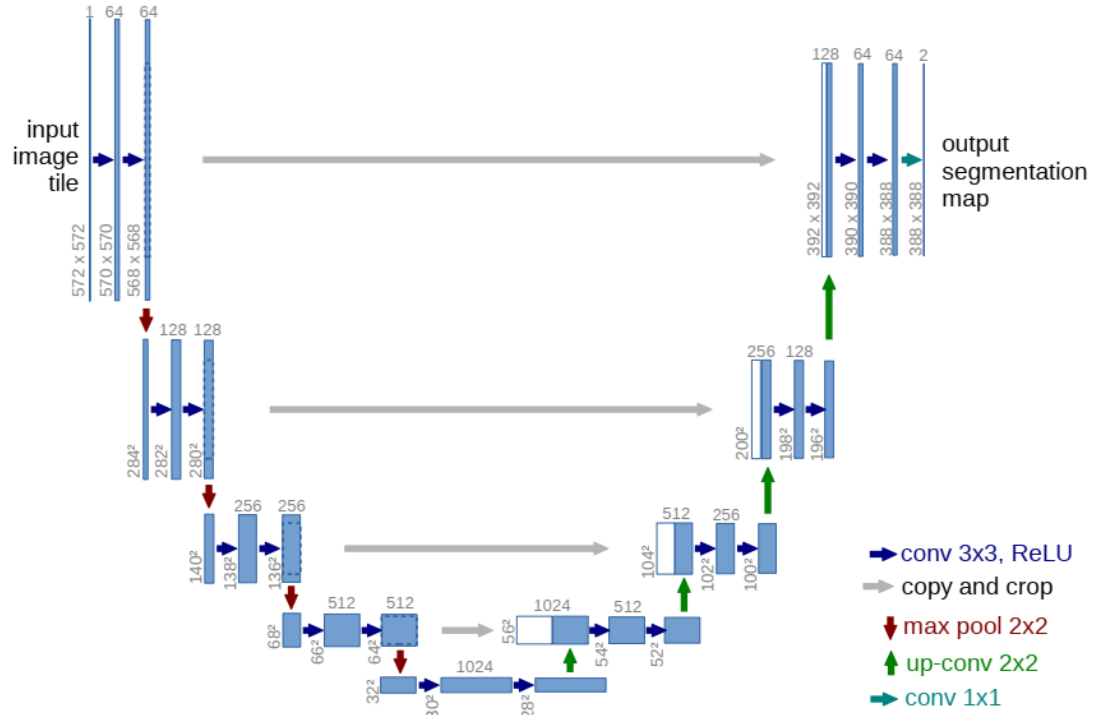


Figure 4.4: U-Net architecture (66)

Conceptual design of the neural architecture that has been designed in this work is based on encoder decoder approach. Useful approach from above mentioned state of the art neural network has been used to design the neural network architecture as explained in next section.

4.3 Neural network architecture

As shown in some of the algorithm above, network must be designed in encoder-decoder format. A neural network consists of different layers as mentioned in chapter 3. Most used layers are two, convolution and pooling layers. Encoder part of the network mainly contains no. of these two layers but for decoder part, we want to do exactly the inverse of what we did in encoder part to get the output which is very close to input. Inverse layers, such as deconv layer for conv layer

and unpooling layer for pooling layer are used in decoder part. As shown in section 3.4.1, one more layer which is very useful for semantic segmentation is skip connection. As information on exact location of a pixel is necessary for better segmentation, this layer plays a good role in transferring location information which is lost by dimension reduction. Dropout layers has been also used while optimizing the network.

We also have to think about computational power and memory usage. Which is a main concern of this work, which leads to a fix amount of layers that can be used at the time of parameter selection for optimization algorithm. Parameter selection affects the outcome the most and, that's why, enough amount of thoughts should be given to select the amount and range of parameter (hyper parameter). In section 3.4.1, information on how different parameters affects the output of network is shown.

An example of basic architecture idea is shown in figure 4.5.

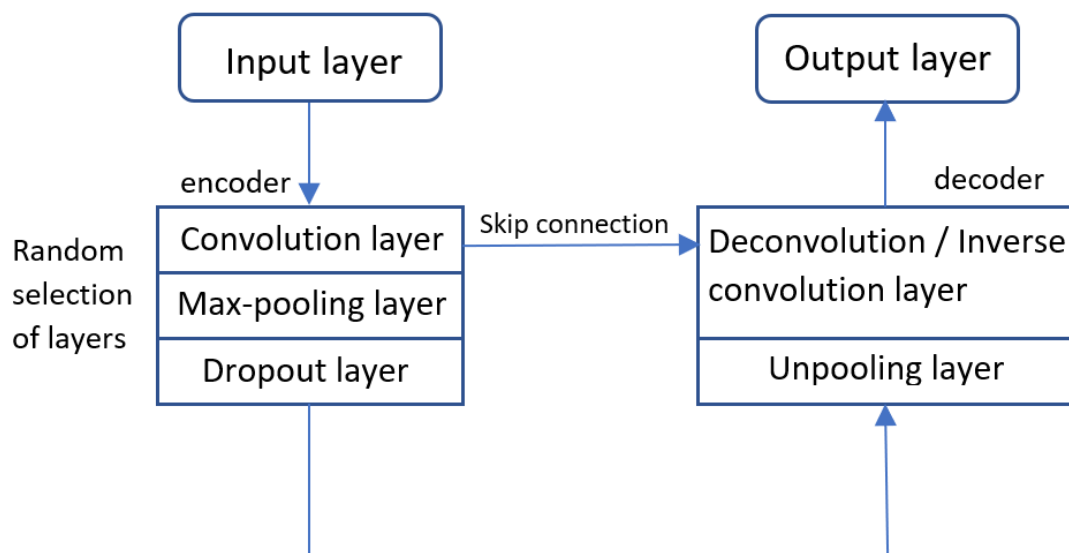


Figure 4.5: Simple conceptual design

4.3.1 Parameter selection

Parameter are selected by considering requirements of the embedded system and segmentation accuracy as described in section 4.1. Two key requirements from those four are, **computational power and memory usage**. Parameter of neural network must be selected considering these two requirements. For these two requirements, the layer that is import, is convolution layer. As discussed in section 3.2.1, at convolution layer, a window of specific length is passed on the whole image and a 2-dimensional activation map is generated.

Not all the layers take same computational power as mentioned in (22). When we compare the number of operation of pooling layer to convolution layer, we can neglect the operation by pooling layer. At convolution layer, most **number of operations** takes place, which can be calculated as below:

$$N_{op} = \frac{I_H * I_W * I_C * K_H * K_W * O_C}{S^2}$$

Where; N_{op} = Total number of operations,

I_H, I_W, I_C = Height, width and channel of input layer,

K_H, K_W = Height and width of the kernel,

O_C = Output channel,

S = Number of strides.

Memory usage also depends on the size of the layers. At the time of segmentation, always two layers will be there in the memory. So at the time of calculating memory usage, two layers must be considered. Max memory used by two layers together must stay less than the required amount at each stage. Memory usage per layer can be calculated by simply multiplying height, width and number of channel of the layer. So memory usage at each stage is calculated as:

$$Memory\ usage = 4 * ((I_H * I_W * I_C) + (O_H * O_W * O_C))$$

Where; I_H, I_W, I_C = Input height, width and channel

O_H, O_W, O_C = Output height, width and channel

If we consider other requirement, segmentation time, it depends on number of operation and memory usage. For number of object, we don't have to change any specific parameter.

Now that requirements have been considered, the parameter we can choose, which will take care of these requirements, is number of layers. Too less layers will decrease the accuracy and too many layers will increase the computation time. As we are going to use optimization method to optimize our network, we can fix a range for now and with the help of optimization algorithm, number and type of layer will be optimized. This applies to all parameter.

As stated earlier in this section, the second criteria which is important is the **segmentation accuracy**. Accuracy depends of many factors. Some of which can be considered essential and can be selected as a parameter to optimize. As stated in section 3.4.1, some of the parameters which can be considered to obtain good output accuracy are Kernel size, feature maps, leaning rate, Dropout layer and skip connection. Their importance, when used for semantic segmentation, is explained a bit.

- **Kernel size**

This parameter is used for convolution network as the size of the window which is passed on the whole image for convolution operation. It is helpful to learn features, such as line,

corner etc. from image. It does increase the total number of parameters. It can be used with more stride to get less operation, but as we will see in next section, while applying Evolutionary Algorithm to optimize the network, the selection of layers is done randomly. Same stride must be used at decoder part so random stride selection can make the network too complex to design.

- **Feature maps**

As discussed in last point, with help of convolution layer, network learn different feature of the image. It does so by using filter with different values. Kernel size defines the size of the filter and feature maps defines number of different filters. Each filter generates a 2D activation map which is a feature map. Optimized number of feature maps must be found to get the balance between computational power and learning of the network, the accuracy.

- **Learning Rate**

Learning rate, as the name suggest, is directly related to the speed at which network learns. As for evolutionary algorithm, the optimum network will be finalized after training each network in each generation, training time can be considered as an indirect parameter too.

- **Dropout layer and skip connection**

Dropout layer helps the network from over-fitting by deactivating some neurons. This helps neurons learn different features. Skip connection is really helpful for semantic segmentation (32), (33) because it helps transferring location information from encoder part to the decoder part.

After selecting starting parameter and conceptual design of the network architecture, to get the optimized networks, an optimization algorithm Evolutionary Algorithm, as mentioned in section 3.4, can be used. But before we implement the optimization algorithm, one more thing left is to prepare training data.

4.4 Data-set generation

The data-set was generated using camera which is in the palm of the KIT prosthetic hand. The camera module, OV2640, is coupled with a Time-of-Flight(ToF) sensor to get the depth information. ToF sensor, VL531X is fast and accurate depth measuring sensor which can measure distance up to 400cm with frequency up to 50Hz, that is enough range when it is used for getting info on objects to be grasped by a prosthetic hand. As for this thesis, effect of depth on the outcome is to be calculated as well, for each photo depth information was also recorded.

Image capturing

For this work DEMO13 object set, which consists of 13 different objects, is used. For each object, 150 to 200 images were captured. To simulate real life situation, objects with uniform background was captured. Images are captured by the on hand camera module and once the whole image frame is created, they are sent via Universal Asynchronous Receiver-Transmitter (UART) to the hard disk. Before the images are saved in the hard drive, they are processed through a python script.

Image was captured at Robotics Lab of KIT. Attention to lightning condition and angle of capturing was given. Though, to simulate real life condition, images were augmented using keras' data augmentation function as shown in section 4.4.2. Also background images were captured using the same camera module. Attention was paid to generate diverse image data-set with different angle and distance.

13 objects from the data-set is shown in figure 4.6 and description of those objects are listed in the table 4.1.

Class ID	Set ID	Object name	Image captured
0	53 (YCB)	Ball	182
1	11 (YCB)	Banana	197
2	60 (KIT)	Bandaaid	195
3	24 (YCB)	Bowl	187
4	10 (KIT)	Canned meat	198
5	1 (YCB)	Chips	190
6	102 (KIT)	Cola	184
7	–	Cup	197
8	64 (KIT)	Fizzy tablets	181
9	48 (KIT)	Hammer	186
10	19 (YCB)	Pitcher	175
11	14 (YCB)	Lemon	176
12	51 (KIT)	Shower gel	177

Table 4.1: Description of DEMO 13 data-set



(a) Ball



(b) Banana



(c) Band-aid



(d) Bowl



(e) Canned meat



(f) Chips



(g) Cola



(h) Cup



(i) Fizzy tablets



(j) Hammer



(k) Pitcher



(l) Lemon



(m) Shower gel

Figure 4.6: DEMO 13 data-set

4.4.1 Pre-processing

Background augmentation

Now to simulate real life situation, some background for these photos were also captured using the same camera module. So that background from the images with object can be easily removed, those images are captured using uniform background. Background to the object is added by removing original background as shown in figure 4.7.

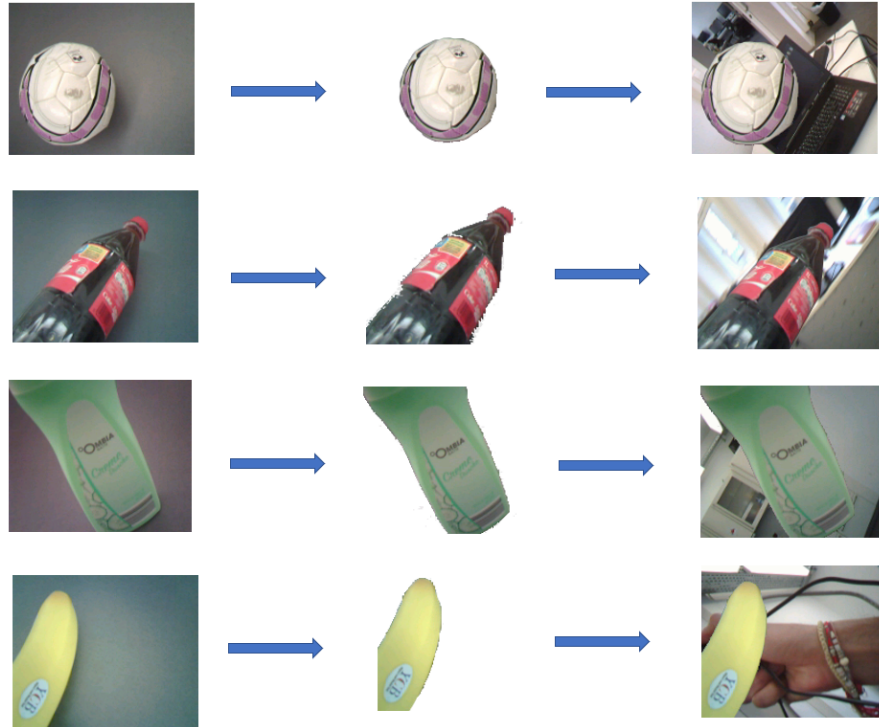


Figure 4.7: From original image (left), background is removed (middle) and new background is added (right)

Binary mask generation

When the background is removed from the original image, the resulting image is used to generate binary mask for that image. For semantic segmentation, mask serves key purpose of providing value of each pixel. For each image, value of 0 (black) given to the pixel which belongs to background and value of 255 (white) is given to the pixel which contains object as shown in figure 4.8.

For semantic segmentation, one more thing which is very necessary is that at the time of training, each pixel of each image must be defined with one value. In this work, each pixel is given value as per the class ID of the object. For example, for ball, each pixel of an image which

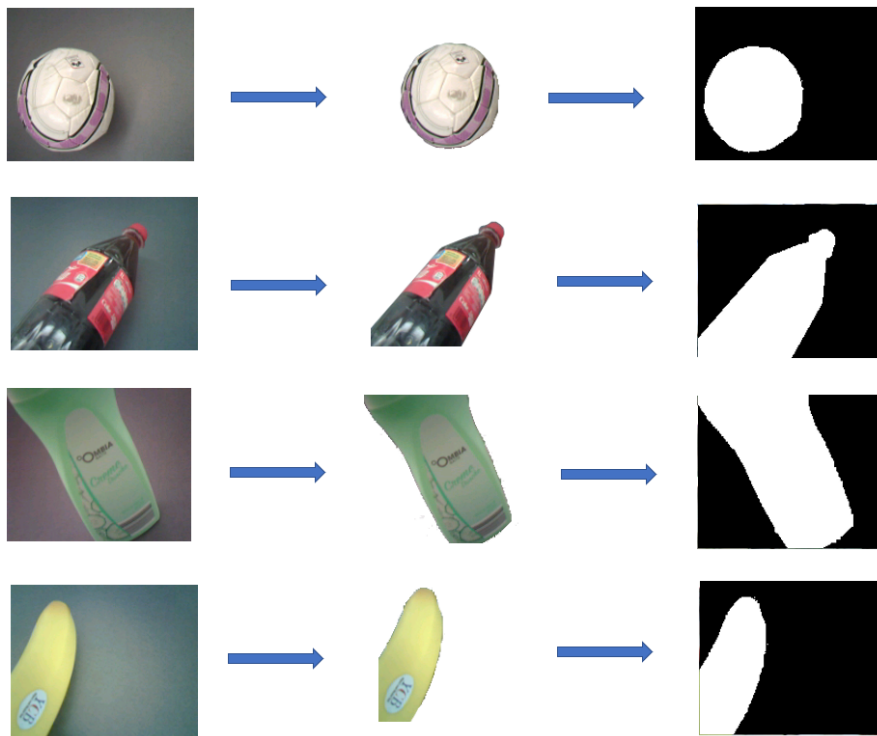


Figure 4.8: From original image (left), background is removed (middle) and binary mask is generated (right)

contains ball is given a value of 0 (Class ID of ball) and each pixel which contains background is given value of 14. Background can also be considered as a class.

4.4.2 Data-augmentation

After finishing pre-processing, images and masks can be used for training of a network. But to increase the data-set size, preprocessed images and masks are augmented using the keras' function. The function which is used is ImageDataGenerator. This function offers many parameters for augmentation. Some of the most useful parameters are mentioned in the sample code below.

```
1 import tensorflow as tf
2
3 tf.keras.preprocessing.image.ImageDataGenerator(
4     rotation_range=0,
5     width_shift_range=0.0,
6     height_shift_range=0.0,
7     brightness_range=None,
8     zoom_range=0.0,
9     horizontal_flip=False,
```

```

10     vertical_flip=False,
11     rescale=None,
12     validation_split=0.0,
13 )

```

Though in this work, depth information is one parameter to take into consideration, augmentation parameters like zoom, rescale are not used. For generating new images, horizontal flip and vertical flips are used. Rotation can also be used but at the time of capturing image that factor is already taken care of. Image and masks are both augmented such that there are no mismatch between augmented images and masks. To make sure that image and mask undergoes same augmentation, same seed value is used. A sample code used to apply vertical and horizontal flip is shown below.

```

1  #imports
2  import os
3  import numpy as np
4  from tensorflow import keras
5  from keras.preprocessing.image import ImageDataGenerator
6
7  #Function to get image and class names.
8  #input image_path: folder containing different object folder
   with images.
9  #     mask_path: same folder as image_path but with mask which
   have same name as images in image_path
10 #     datagen: keras.ImageDataGenerator object
11
12 def data_prep(folder_path, datagen):
13     seed = 18
14     image_path = os.path.join(folder_path, 'Images')
15     mask_path = os.path.join(folder_path, 'Masks')
16     for class_name in os.listdir(image_path):
17         for image_name in os.listdir(os.path.join(image_path,
   class_name)):
18             image = np.expand_dims(load_img(os.path.join(
   image_path, class_name, image_name)), 0)
19             save_dir = os.path.join(folder_path, 'All_images')
20             image_name = image_name.split('.')[0]
21
22             datagen.fit(image)
23
24             for x, val in zip(datagen.flow(image,
   #image to be augmented
25                 save_to_dir=save_dir,
   #save destination

```

```

26         save_prefix=f'{class_name}_{image_name}',
        #save each image with prefix as 'image_name_random
number'
27         seed=seed,
        #same augmentation for image and mask
28         save_format='png'), range(3)) :
        #no. of augmented image from each image
29         pass
30
31     for class_name in os.listdir(mask_path):
32         for image_name in os.listdir(os.path.join(mask_path,
class_name)):
33             image = np.expand_dims(load_img(os.path.join(
mask_path, class_name, image_name)), 0)
34             save_dir = os.path.join(folder_path, 'All_maskst')
35             image_name = image_name.split('.')[0]
36
37             datagen.fit(image)
38
39             for x, val in zip(datagen.flow(image,
#image to be augmented
40                 save_to_dir=save_dir,
        #save destination
41                 save_prefix=f'{class_name}_{image_name}',
        #save each image with prefix as 'image_name_random
number'
42                 seed=seed,
        #same augmentation for image and mask
43                 save_format='png'), range(3)) :
        #no. of augmented masks from each mask
44         pass
45
46 datagen = ImageDataGenerator(
47     horizontal_flip=True,
48     vertical_flip=True)
49 folder_path = r'..\..\DataRecording\Image_processing'
50 data_prep(folder_path, datagen)

```

With the help of this function, three different images from one single image is generated. Though each images have different names, image and mask that undergoes same augmentation have same names. That helps a lot when using them at the time of training. Augmentation helps network learn by providing more data. Output of the function defined above is shown in figure 4.9.

After having all the data for training, images and masks are split into test and train data as 80 - 20. 80% images are used in training and 20% are used as testing data to evaluate the trained model. From training data, again, 80% are used for training the network and 20% are used for validation purpose. Summery of the process from capturing the image till dividing into different

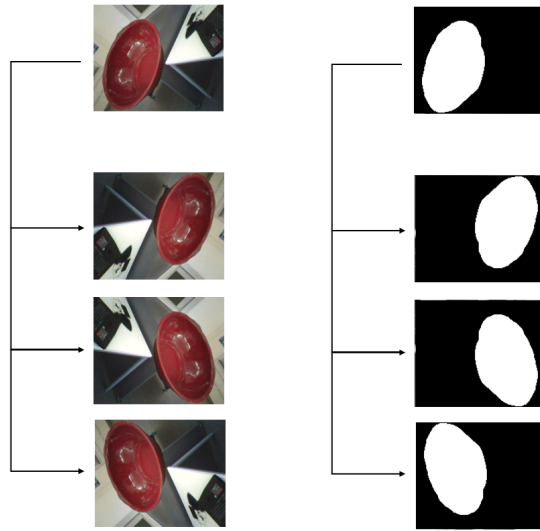


Figure 4.9: Same augmentation is applied to image and mask to avoid mismatch

set is explain via a flow chart diagram in figure 4.10.

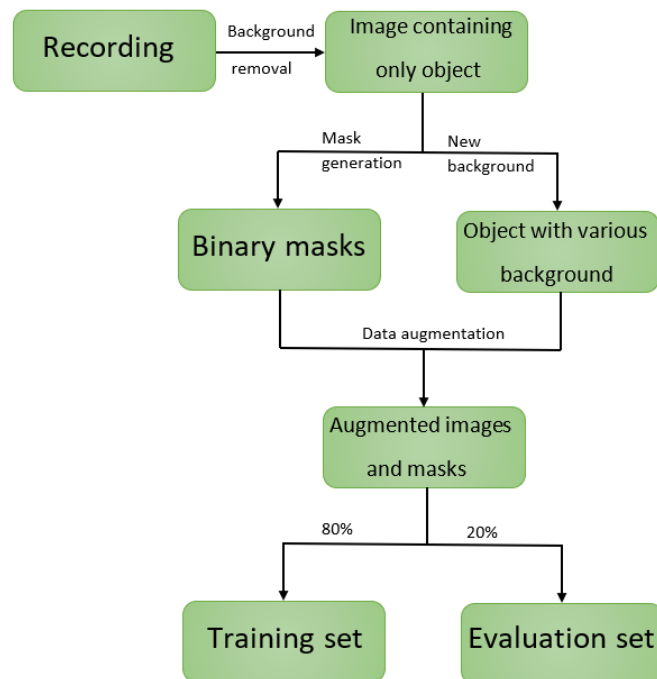


Figure 4.10: Image processing, from capturing to creation of training and evaluation set

4.5 Evolution

Now that all those images and masks are ready and divided into training and evaluation set, training of the network can be done. Having some parameters and basic structure, evolution algorithm can automatically optimize the network. As we discussed earlier, all the parameters and a basic structure of network (input, output) must be defined. All the parameters, such as Number of layers, Kernel size, Feature maps, Learning rate etc., are given as a python dictionary.

Selection of layer is random and limit on pooling layer is given to avoid situation where the dimension of the image doesn't match at the decoder stage. To explain it a bit, pooling layers helps reducing the dimension of the image. To be exact it divides it into half each time. Now if the dimension doesn't belong to a value of 2^n , at one point it can become a value that is not dividable by 2. If that happens then at the time of decoder part, we can't get the same shape as in the encoder part.

For example, if dimension is 90 at some point, then a pooling layer will convert it to 45 and one more to 22. Now at the decoder part, 22 will be increased to 44 and not 45. Which creates problem for concatenate layers too.

As output softmax function for 14 class is defined. 13 object classes and 14th background class.

4.5.1 Implementation of the algorithm

Algorithmus 1 : Pseudocode for evolutionary algorithm

1 EA Optimization function (P_i : Population)

Input : P_i : Population of network each generation, r : retain rate, $\hat{r}$: random retain rate,

μ : mutation rate

Output : P_{i+1} : Population for next generation

2 $P_i \leftarrow$ network training, save fitness value

// Sorting according of P_i according to fitness value

3 $P_{parents} \leftarrow$ choose some of the best from P_i , selection according to r & $\hat{r}$

4 $P_{child} \leftarrow$ generate children from $P_{parents}$

5 $P_{child} \leftarrow$ create child based on mutation as per μ

6 $P_{i+1} \leftarrow P_{parents} \cup P_{child}$

Algorithm is designed in a way that it selects one parameter value for each parameter and use it to generate starting population. At each generation, some of the networks with best fitness function is selected and new population is then generated from the parameters of those best networks.

Pseudocode describing the process is described in listing 1.

Training

Training is done in Keras with Tensorflow as backend. Training can be done till the convergence is met. That can be assure by using Tensorflow callback function *tf.keras.callbacks.EarlyStopping*. Early stopping criteria can be set as a delta value. If the accuracy of the model won't change with a minimum delta, for specified epochs, training will stop on its own.

In this work though, networks are trained for 30 epochs and then the best networks are trained till convergence to get the final result. For 30 epochs also, one network was taking around 30 to 35 minutes. So for total 10 generation and 10 networks per generation are trained, it takes almost 52 hours to train all the networks.

Sorting

After each generation, networks are sorted according to the fitness function. Fitness function shows how good this network is for defined condition. Only accuracy is not enough if other requirements are considered.

For this work, fitness function is based on three main parameters: The segmentation accuracy, Number of floating point operation, Memory usage.

Accuracy is calculated on evaluation set. Number of floating point operations and memory usage is calculated as per the formula mentioned in section 4.3.1. To calculate number of operations, only convolution layers are considered. To calculate memory usage, two layers at a time are considered.

Fitness function for number of operation and memory usage is defined by general logistic function. The fitness function is calculated with the formula defined below.

$$f(\alpha, \beta_{conv}, \gamma) = \alpha + (1 + e^{\beta_{conv} - (1 + l_o)})^{\left(-\frac{1}{2 + l_o}\right)} + (1 + e^{\gamma - (1 + l_m)})^{\left(-\frac{1}{2 + l_m}\right)}$$

where, l_o and l_m = limit for max. allowed operations and memory respectively,

β_{conv} = Total number of operations,

γ = Max. memory usage

It has been designed in a way that

$$0.0 < f(\alpha, \beta_{conv}, \gamma) < 3.0$$

Though initial tests showed that limit of 2 million for number of operations and memory usage limit of 8×10^5 for 14 classes is too low. For semantic segmentation, at the last layer, we get the image of original dimension as an output, which increase the value of both the parameters (no. of operation and memory usage). This function can be used for different limits of max allowed valued of number of operation and memory usage. As shown in formula below, value of the unknown can be changed to get fitness function for different max number of operation and memory usage.

Parents selection

As to generate population for next generation, network which performed well are selected. Selection of number of parents depends on the r (retain rate) and $\hat{r}$ (random retain rate). At the time of initial calculation, around 40% networks are selected as parents for each generation and then child were created from them. Sometimes random network which didn't perform well is also selected as per $\hat{r}$ and added to the list of networks for next generation.

Child and Mutation

At the time of creating child from parents, first of all, two different networks are selected and then parameters from random parent is selected as the parameter for the child. As for mutation, the parameter values which is different from either parent is chosen and given to the child network.

For this implementation, selected parameters are: Number of layers, Kernel size, Feature Map, Dropout layer, Skip connection, Learning rate, Batch size.

To create child without mutation, keys from the dictionary of parameters is randomly selected and the value of the key from either parent is selected and given to the child. Where with mutation, the value of parameter is from main dictionary is selected and not from either parent.

Parameters are selected from the dictionary defined below.

```

1 para_choice = {'Layers': [5, 7, 9],
2               'Filter_Size': [[1,3,5,7], [1,3] , [1,5] , [1,7],
3                               [3,5], [3,7], [1,3,5], [1,5,7]],
4               'Dropout': [True, False],
5               'Skip': [True, False],
6               'Learning_Rate' : [0.001, 0.005, 0.001, 0.01,
7               0.02],
               'Batch_size' : [3,4,5,6,7,8]}

```

Results after applying evolutionary algorithm to create network for different hardware is presented in next chapter.

Chapter 5

Evaluation

In this chapter, final result of optimized neural network has been discussed. Optimized network is designed and results for three types of hardware have been discussed 5.2. Cortex-M7 micro-controller 5.2.1, which is currently used in hand, Artix-7 FPGA (XC7A50T-1CSG324C) 5.2.2 and Nvidia Jetson TX1 5.2.3. Top five network for each hardware have been mentioned and top network has been described in details. To measure the impact of depth information, micro-controller architecture has been refined a bit as shown in section 5.3 and evolutionary algorithm has been then applied to that architecture to find optimized network.

5.1 Influence of augmentation

As a starting stage, evolutionary algorithm was used to find optimal network with images which were obtained after background augmentation (section 4.4.1). Those networks were then compared with the network which were trained with images obtained after using keras' *DataAugmentation* function. Use of this function has been explained in section 4.4.2.

Accuracy of top 5 networks for images with *DataAugmentation* function were higher than those with only background images. *DataAugmentation* function provides more images and also with different angles and position, which helps network to learn efficiently. As shown in figure 5.1, accuracy increase of almost 2 to 2.2% can be seen with augmentation.

More image data tends to increase the robustness of the network. It doesn't affects the training accuracy only but as shown in 5.1, it also increases the evaluation accuracy. By using *DataAugmentation* function, brightness can also be varied, which can be considered as different illumination condition. This is essential for any network. When real-life condition is simulated, different illumination is a normal situation. Thus data augmentation is essential for increasing the

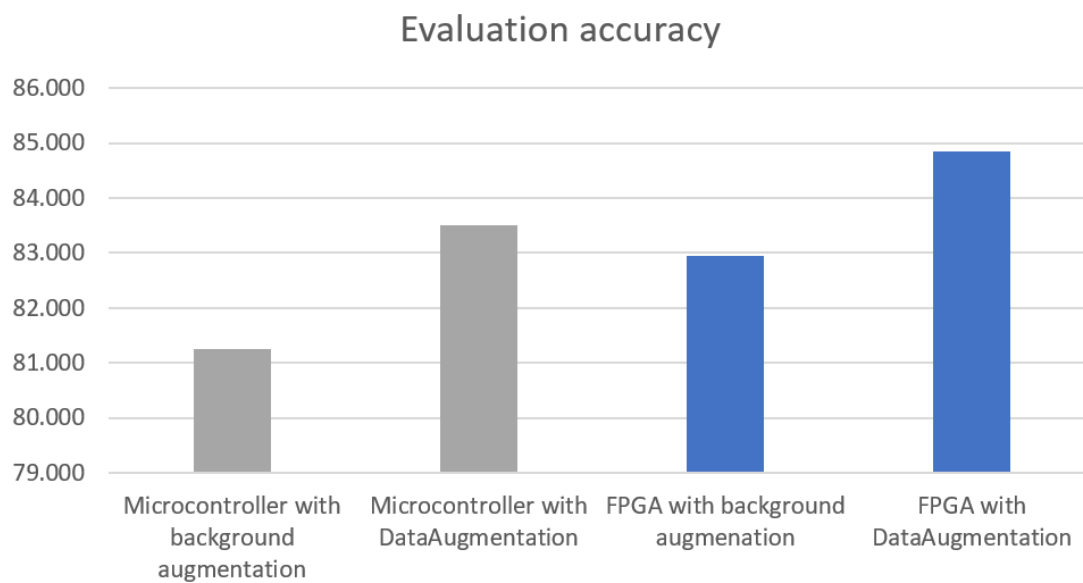


Figure 5.1: Effect of data augmentation on evaluation accuracy for micro-controller (gray) and FPGA (blue)

efficiency and robustness of any network.

After computing the effect of augmentation, networks are trained and evaluated with images which are obtained with *DataAugmentaion* function of keras. Network for three different hardware are evaluated and compared as presented in next section.

5.2 Results of Evolutionary Algorithm

Evolutionary algorithm with 15 initial population and 10 generation has been implemented to get optimized network for each hardware. As for fitness function number of operations and memory usage have been considered. Values of these two parameters and the evaluation accuracy for each population have been collected. Result for each hardware is represented in this section.

5.2.1 Result for micro-controller

To apply evolutionary algorithm (EA), we first have to find out two values for fitness function, max. allowed number of operation per second and max. allowed memory usage. For micro-controller, some work has been already done and the limit of number of operation has been already decided. To get execution time of 120ms, allowed max number of operation per second is 2M. Micro-controller consists of 1MB RAM, which is 1MB. Memory usage limit is thus decided as 0.8MB.

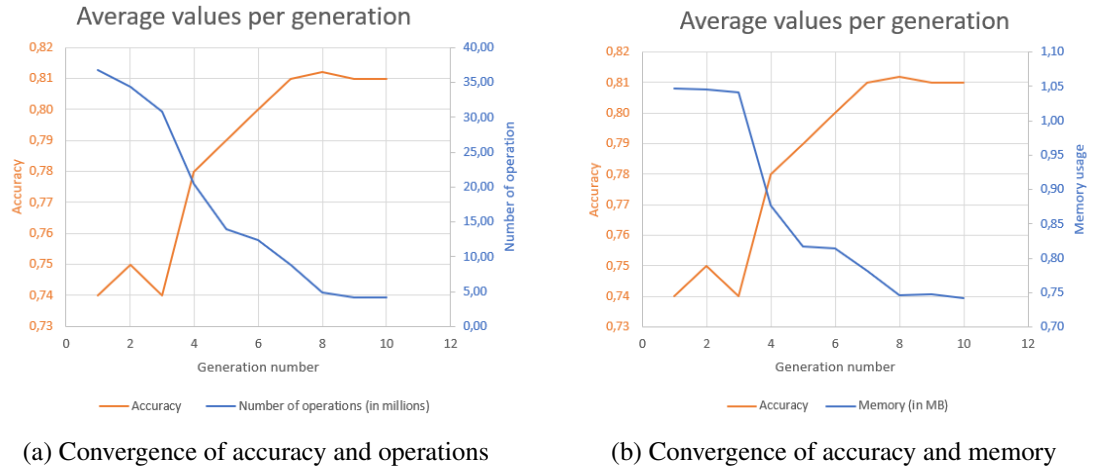


Figure 5.2: Convergence of accuracy, number of operation and memory usage in 10 generations for micro-controller

Initial results showed that even after 10 generation the number of parameters are too high. To avoid that problem, at the start of the network, max-pooling layers with kernel size of 2 and 4 has been used to reduce the dimension at the start. This helps in reducing the number of operation and also memory usage, but it affects accuracy of the network.

Convergence of accuracy, number of operation and memory usage

For micro-controller, as shown in figure 5.2, after 7th generation accuracy doesn't seem to change much whereas number of operation reaches stability at 8th generation. Same as number of operation, memory usage doesn't deviate much after 8th generation.

Each network was trained for 30 epochs. On a NVIDIA TITAN X this took almost half hour for one population, so almost 7.5 hours per generation.

Maximum allowed number of operation is 2M but as we can see from the figure 5.2, average number of operation of last generation is around 5 millions. As we will see in next section, even if the average of number of operations is high, top 5 networks have less number of operations. For memory usage, 1 MB RAM is available, which is 1M bytes. So max. limit has been considered as 0.8MB. At last generation we get average around 0.75MB, which is under limit.

Analysis of best Architectures

Table 5.1 shows some of the best optimized architecture at last generation. As can be seen from those best networks, maximum accuracy of 0.83 has been achieved. Number of operations are also below limit for almost all networks. Memory usage is also below 0.8MB except some networks at the bottom.

	Fitness	Accuracy	Ops. (in M)	Memory (in MB)	Selected Parameters (L_R = Learning rate) (K_S = Kernel size)	Network Layers (Conv[kernel x Feature maps]) (Pool[kernel x channel])
1	2.805	0.826	1.429	0.6082	{'Layers': 9, 'K_S': [5, 1, 3], 'Dropout': True, 'Skip': False, 'L_R': 0.0005, 'Batch_Size': 8}	Pool4 [4x3] Conv[1x3] Pool[2x3] Conv [3x5] Conv[1x10] Deconv[5x8] Unpool [2x8] Deconv [3x5] Unpool[4x5]
2	2.797	0.834	2.117	0.6082	{'Layers': 9, 'K_S': [5, 1, 3], 'Dropout': False, 'Skip': False, 'L_R': 0.02, 'Batch_Size': 8}	Pool4[4x3] Conv[1x3] Conv[3x5] Pool [2x5] Conv[1x10] Deconv [3x8] Unpool[2x8] Deconv [5x5] Unpool[4x5]
3	2.796	0.817	1.551	0.4055	{'Layers': 7, 'K_S': [1,3, 7], 'Dropout': False, 'Skip': False, 'L_R': 0.0005, 'Batch_Size': 6}	Pool4[4x3] Conv[1x3] Conv[7x5] Conv[1x10] Deconv [1x5] Deconv [3x3] Unpool[4x3]
4	2.793	0.821	1.634	0.7096	{'Layers': 7, 'K_S': [1, 3], 'Dropout': False, 'Skip': False, 'L_R': 0.02, 'Batch_Size': 6}	Pool[2x3] Conv [1x3] Conv [3x3] Conv [1x5] Deconv [3x3] Deconv [1x6] Unpool[2x6]
5	2.793	0.816	1.571	0.6082	{'Layers': 7, 'K_S': [1, 3,7], 'Dropout': False, 'Skip': False, 'L_R': 0.01, 'Batch_Size': 6}	Conv[1x3] Pool[2x3] Conv [1x5] Conv[1x7] Deconv[3x3] Unpool[2x3] Deconv[1x5]
...		...	...	...	...	...
15	2.707	0.806	3.136	0.8110	{'Layers': 5, 'K_S': [1,5], 'Dropout': False, 'Skip': False, 'L_R': 0.02, 'Batch_Size': 6}	Pool[2x3] Conv[5x5] conv [1x10] Deconv [1x7] Unpool[2x7]

Table 5.1: Best network for micro-controller after 10 generations

For network parameters, network tends to go for high number layers and layers like Dropout and Skip connection are not selected for almost all network. This explains that for this small network, dropout layer works negatively. It thin out layers and thus reduces the accuracy. Where as skip connection increases the number of channel, which in turn, increases the memory usage and number of operations.

Network layers shows that pooling layer with 4 kernel size has been selected as first layer, which explains the low operation number. This layer reduces the dimension of the image width and height by $1/4^{th}$.

Best suited network according to fitness function has been described in detail in table 5.2.

Layer Type	Filter Shape	Output Shape	Ops	Memory
Input	N.A.	$144 \times 176 \times 3$	N.A.	0.1013 MB
MaxPooling	4×4	$36 \times 44 \times 3$	–	0.1203 MB
Convolution	$1 \times 1 \times 3$	$36 \times 44 \times 3$	14.256 k	0.0442 MB
MaxPooling	2×2	$18 \times 22 \times 3$	–	0.221 MB
Convolution	$3 \times 3 \times 5$	$18 \times 22 \times 5$	53.460 k	0.0126 MB
Convolution	$1 \times 1 \times 10$	$18 \times 22 \times 10$	19.800 k	0.0237 MB
Deconvolution	$5 \times 5 \times 8$	$18 \times 22 \times 8$	0.79 M	0.0285 MB
Unpooling	2×2	$36 \times 44 \times 8$	–	0.0633 MB
Deconvolution	$3 \times 3 \times 5$	$36 \times 44 \times 5$	0.57 M	0.0823 MB
Unpooling	4×4	$144 \times 176 \times 5$	–	0.50385 MB
Softmax	N.A.	$144 \times 176 \times 14$	–	0.60825 MB
			1.44M (Total)	0.6082 MB (Max.)

Table 5.2: In detail description of the best network for Micro-controller

The best architecture with filter size and image dimension at each stage has been visualized in figure 5.3.

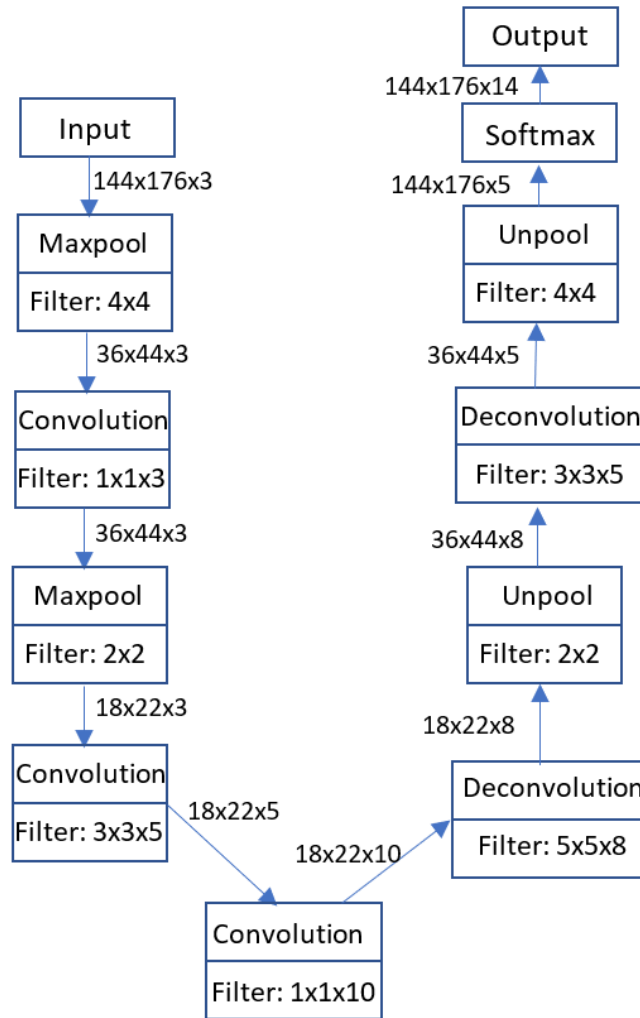


Figure 5.3: Visualization of the best architecture with filter and image size at each step (micro-controller)

Segmentation result

Now that we have best suited network, it has been used to segment images which are not used in training. Results of which can be seen in figure 5.4.

As can be seen from the segmentation result, network can predict true object but with some noise.

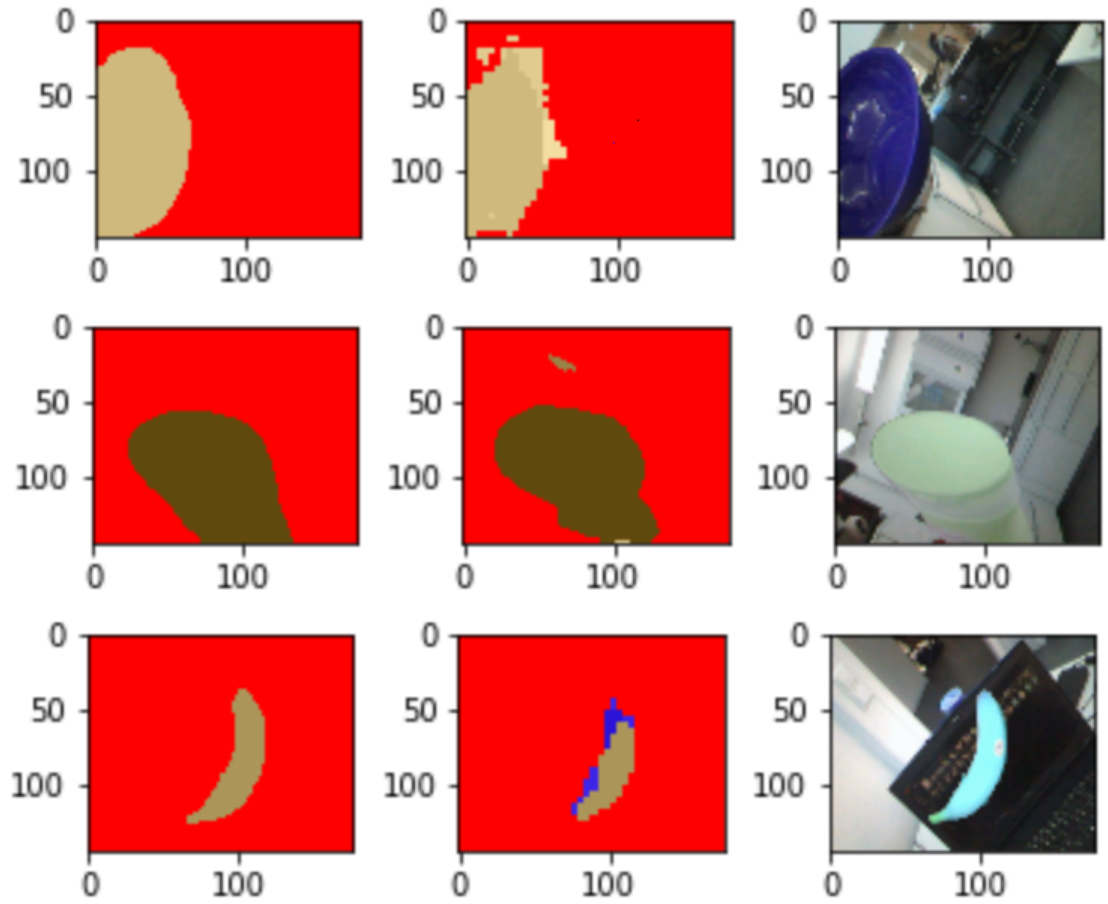


Figure 5.4: Visualization of segmentation Ground truth (left), Predicted output (middle), Image (right) for micro-controller

5.2.2 Result for FPGA

For FPGA, available memory is 2000kbit which is around 0.25MB, which is $1/4^{th}$ of allowed memory for micro-controller. So for FPGA the limit of maximum allowed memory usage has been taken as 0.2MB.

FPGA consist of 120 DSPs at 100MHz frequency. So we get $120 \times 100 = 1.2$ GOPS (Giga operation per second). If we take execution time of 120ms, number of operation allowed per second can be calculated as $1.2G \times 0.12 = 144M$ operations. Maximum limit for number of operation is thus considered as 80M.

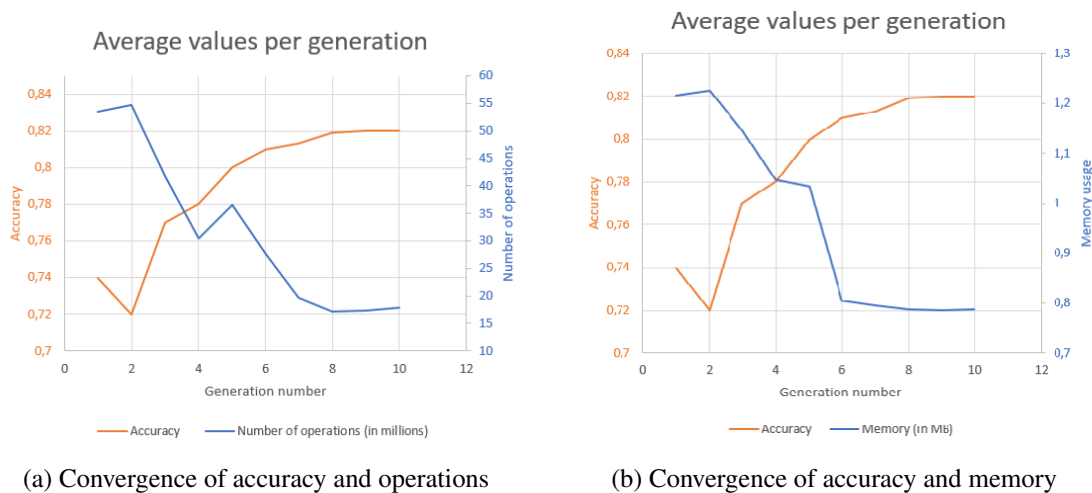


Figure 5.5: Conversion of accuracy, number of operation and memory usage in 10 generations for FPGA

Each network for all generations has been trained for 30 epochs. It also took 30 minutes for one network to train.

Convergence of accuracy, number of operation and memory usage

For FPGA, as seen from the figure 5.5, number of operations decreases fast in starting generations and at last generation it averages at around 17 million operations, which is less than allowed operations. But the problem lies with memory usage for FPGA as mentioned in next section.

But the average memory usage in last generation is still around 0.8MB, which is more than allowed number. As discussed for micro-controller, at output layer, two layers with full dimensions are together, which causes too much memory usage.

Analysis of some of the best architectures

Table 5.3 shows some of the best optimized architecture at last generation. As can be seen from those best networks, maximum accuracy of 0.843 has been achieved. Number of operations are also below limit for almost all networks. One thing to notice from the results is that for FPGA all the networks have more memory consumption than allowed.

As memory consumption depends on the image size, only way to decrease the memory usage to allowed limit is to decrease the original size of the image. Maxpooling layer doesn't help here because for image segmentation, at last stage we have one layer with full dimension with some

number of channels. Only that one layer is enough to cross this limit.

After 10^{th} generations, total 15 best suited networks have been found. Layer structure, number of operations and memory usage for each network has been mentioned and they have been sorted according to the fitness value, as shown in table 5.3. The first of those network has been explained in detail in table 5.4.

The best architecture with filter size and image dimension at each stage has been visualized in figure 5.6.

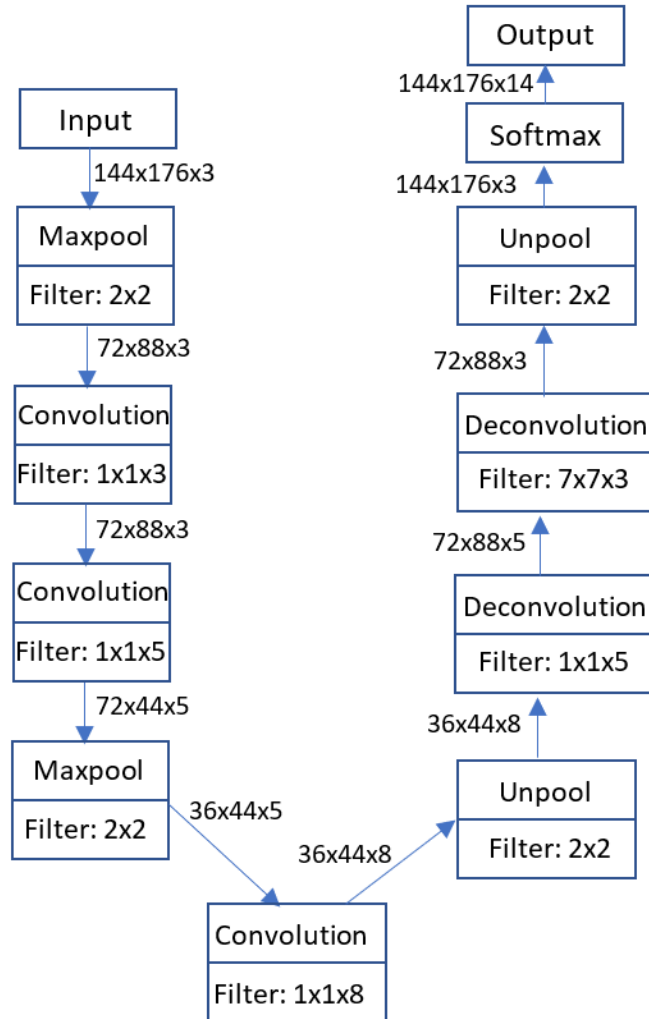


Figure 5.6: Visualization of the best architecture with filter and image size at each step (FPGA)

	Fitness	Accuracy	Ops. (in M)	Memory (in MB)	Selected Parameters (L_R = Learning rate) (K_S = Kernel size)	Network Layers (Conv[kernel x Feature maps]) (Pool[kernel x channel])
1	2.675	0.843	5.125	0.4055	{'Layers': 9, 'K_S': [1,7], 'Dropout': False, 'Skip': False, 'L_R': 0.001, 'Batch_Size': 6}	Pool [2 x3]Conv[1x3] Conv [1x5] Pool[2x5] Conv[1x8] Unpool [2x8] Deconv [1x5] Deconv[7x3] Unpool[2x3]
2	2.674	0.839	2.274	0.4055	{'Layers': 7, 'K_S': [1, 3], 'Dropout': False, 'Skip': False, 'L_R': 0.02, 'Batch_Size': 7}	Pool[2x3] Conv [3x3] Conv [3x5] Conv [1x8] Deconv [1x5] Deconv [3x3] Unpool[2x3]
3	2.669	0.834	4.096	0.4055	{'Layers': 7, 'K_S': [1,3,5,7], 'Dropout': False, 'Skip': False, 'L_R': 0.001, 'Batch_Size': 7}	Pool[2x3] Conv[1x3] Pool[2x3] Conv[1x10] Unpool [2x10] Deconv [3x7] Unpool[2x3]
4	2.412	0.836	13.924	0.6082	{'Layers': 9, 'K_S': [1,7], 'Dropout': False, 'Skip': False, 'L_R': 0.02, 'Batch_Size': 6}	Pool[2x3] Conv[1x3] Pool[2x3] Conv [7x5] Conv[1x10] Deconv [1x8] Unpool[2x8] Deconv [7x5] Unpool[2x5]
5	2.403	0.828	5.520	0.6082	{'Layers': 9, 'K_S': [1, 5], 'Dropout': False, 'Skip': False, 'L_R': 0.01, 'Batch_Size': 4}	Pool[2x3] Conv[5x3] Pool[2x3] Conv [5x5] Conv[1x10] Deconv[5x8] Unpool[2x8] Deconv[1x5] Unpool[2x5]
...		...	...	...	...	...
15	1.900	0.786	0.0142	0.12672	{'Layers': 7, 'K_S': [1,7], 'Dropout': True, 'Skip': False, 'L_R': 0.02, 'Batch_Size': 6}	Pool[2x3] Pool[2x3] Pool[2x3]Conv[1x10] Unpool[2x10] Unpool[2x10] Unpool[2x10]

Table 5.3: Best network for FPGA after 10 generations

Layer Type	Filter shape	Output shape	Ops	Memory usage
Input	N.A.	144x176x3	N.A.	304.128 kB
MaxPooling	2x2	72x88x3	–	380.160 kB
Convolution	1x1x3	72x88x3	57.024 k	152.064 kB
Convolution	1x1x5	72x88x5	95.040 k	202.752 kB
MaxPooling	2x2	36x44x5	–	158.400 kB
Convolution	1x1x8	36x44x8	63.360 k	82.368 kB
Unpooling	2x2	72x88x8	–	253.440 kB
Deconvolution	1x1x5	72x88x5	253.440 k	329.472 kB
Deconvolution	7x7x3	72x88x3	4.656 M	202.752 kB
Unpooling	2x2	144x176x3	–	380.160 kB
Softmax	N.A.	144x176x14	–	405.504 kB
			5.125 M (Total)	405.504 kB (Max.)

Table 5.4: In detail description of the best network for FPGA

Segmentation result

In table 5.4 mentioned network has been used to visualize the segmentation result. Results shows some noise in the segmentation. Though this model separates the object from background better than the model of micro-controller, it still causes mistake in identifying some part of the object and mixes two objects.

Results are shown in figure 5.7.

So for FPGA, although accuracy is more than micro-controller, because of the limit of allowed memory usage, it is not possible to run semantic segmentation on KIT hand with Artix-7 FPGA (XC7A50T-1CSG324C).

One way to deal with this problem is to decrease the dimensions of the original image at the input layer. By doing that at the output layer the image can be fit in the memory. The output can then be directly resized to the original size without much computation requirement.

5.2.3 Results for NVIDIA Jetson TX1

NVIDIA Jetson TX1 is a embedded hardware which is designed to perform well for neural network. As per official documents from NVIDIA, it can perform up to 1TFLOPS (Tera floating point operations per second). MAC (multiply-and-accumulate) operation consists of one multiplication and one addition number whereas floating point operation can be considered as one multiply or one addition operation. So when considering limit for number of operations, 0.5 TFLOPS can be taken as the max number of operations for the system. So if we consider 120ms execution time, $0.5T \times 0.120 = 60M$ can be taken as the limit for max allowed operations.

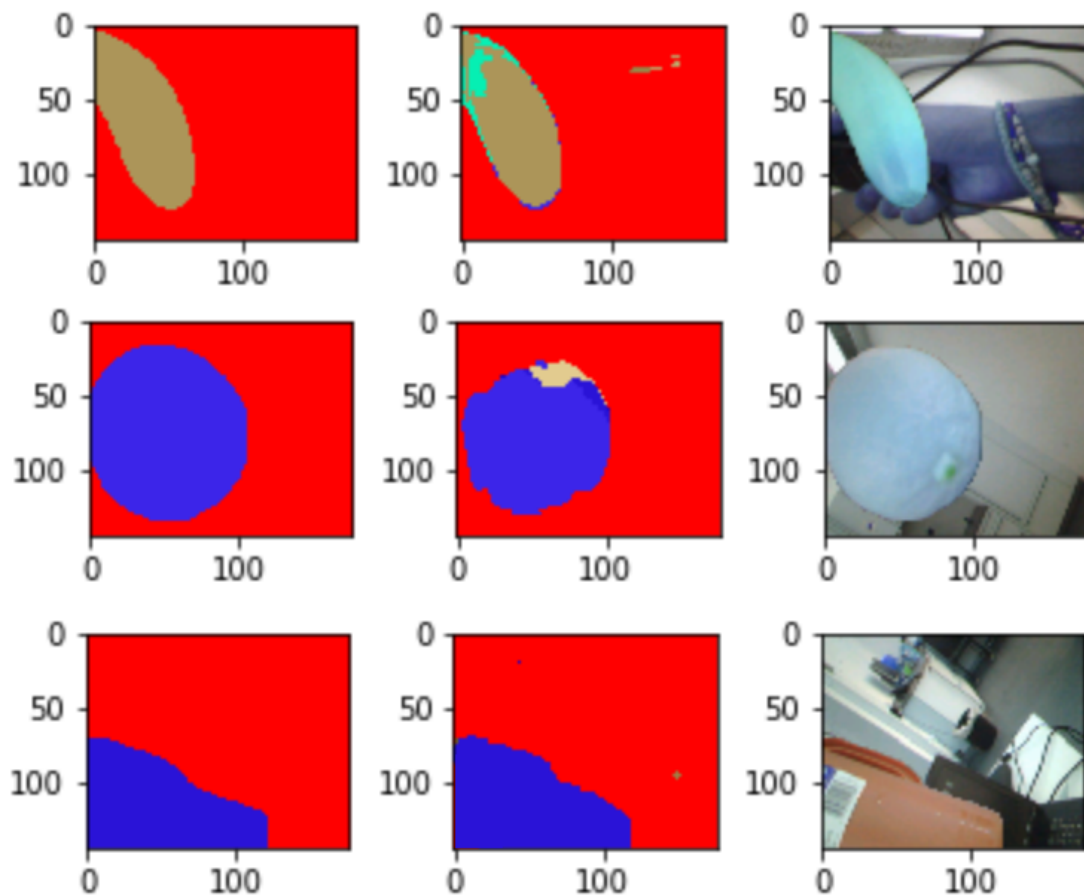


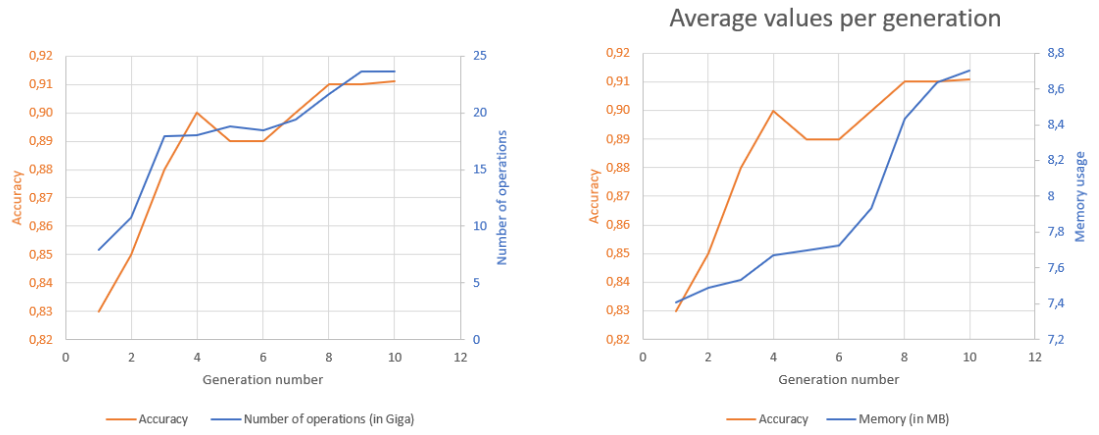
Figure 5.7: Visualization of segmentation Ground truth (left), Predicted output (middle), Image (right) for FPGA

This hardware supports 4GB memory, so for fitness function, 2GB has been taken as the max limit.

Each network for all generations has been trained for 40 epochs. It also took 40 minutes for one network to train.

Convergence of accuracy, number of operation and memory usage

For Jetson TX1, as seen from the figure 5.8, number of operations and memory keeps increasing till last generation but the accuracy converges to average of around 0.91. For Jetson TX1, while training network, parameter dictionary has been changed according to the available resource. Max number of layers are increased to 13 from 9 and also feature maps values have been adjusted according to number of layers. Max feature map value has been increased to 64 from 10.



(a) Convergence of accuracy and operations

(b) Convergence of accuracy and memory

Figure 5.8: Conversion of accuracy, number of operation and memory usage in 10 generations for NVIDIA Jetson TX1

Convergence rate can be seen from figure 5.8.

Analysis of some of the best architectures

Table 5.5 shows some of the best optimized architecture at last generation. Maximum accuracy of 0.939 has been achieved for the best suited network. As for number of operation and memory usage, they keep increasing till last generation and stays below max. allowed limit.

	Fitness	Accuracy	Ops. (in M)	Memory (in MB)	Selected Parameters (L_R = Learning rate) (K_S = Kernel size)	Network Layers (Conv[kernel x Feature maps]) (Pool[kernel x channel])
1	2.938	0.9394	91.939	1.622	{'Layers': 13, 'K_S': [3,5], 'Dropout': True, 'Skip': False, 'L_R': 0.005, 'Batch_Size': 7}	Conv[3x8] Pool[2x8] Pool[2x8] Conv[3x8] Conv [3x16] Conv[5x16] Conv[1x32] Deconv [5x16] Deconv [5x16] Deconv[3x8] UnPool[2x8] Unpool[2x8] Deconv[5x8]
2	2.935	0.9360	93.364	1.622	{'Layers': 13, 'K_S': [1,3,7], 'Dropout': True, 'Skip': False, 'L_R': 0.001, 'Batch_Size': 6}	Conv[3x8] Pool[2x8] Pool[2x8] Conv[3x8] Conv [5x16] Conv[5x16] Conv[1x32] Deconv [5x16] Deconv [5x16] Deconv[5x8] UnPool[2x8] Unpool[2x8] Deconv[3x8]
3	2.934	0.9358	99.482	1.622	{'Layers': 13, 'K_S': [1,3,5], 'Dropout': True, 'Skip': True, 'L_R': 0.001, 'Batch_Size': 7}	Conv[5x8] Pool[2x8] Pool[2x8] Conv[3x8] Conv [5x16] Conv[3x16] Conv[1x32] Deconv [3x16] Deconv [5x16] Deconv[5x8] UnPool[2x8] Unpool[2x8] Deconv[5x8]
4	2.933	0.9341	82.438	2.763	{'Layers': 13, 'K_S': [1, 3,7], 'Dropout': False, 'Skip': False, 'L_R': 0.02, 'Batch_Size': 4}	Conv[3x8] Conv[3x8] Pool[2x8] Pool[2x8] Conv [3x16] Conv[3x16] Conv[1x32] Deconv [5x16] Deconv [3x16] UnPool[2x16] Unpool[2x16] Deconv[3x8] Deconv[5x8]
...		...	...	...	...	...
15	2.881	0.8826	86.284	2.763	{'Layers': 13, 'K_S': [1,3,7], 'Dropout': True, 'Skip': False, 'L_R': 0.02, 'Batch_Size': 6}	Conv[3x8] Conv[1x8] Pool[2x8] Pool[2x8] Conv [3x16] Conv[1x16] Conv[1x32] Deconv [1x16] Deconv [3x16] UnPool[2x16] Unpool[2x16] Deconv[1x8] Deconv[1x8]

Table 5.5: Best network for Jetson TX1 after 10 generations

Layer Type	Filter shape	Output shape	Ops	Memory usage
Input	N.A.	144x176x3	N.A.	304.128 kB
Convolution	3x3x8	144x176x8	5.4743 M	1.1151 MB
MaxPooling	2x2	72x88x8	–	1.0137 MB
MaxPooling	2x2	36x44x8	–	253.440 kB
Convolution	3x3x8	36x44x8	912 k	101.376 kB
Convolution	3x3x16	36x44x16	1.8817 M	152.064 kB
Convolution	5x5x16	36x44x16	10.1376 M	304.128 kB
Convolution	1x1x32	36x44x32	811 k	456.192 kB
Deconvolution	5x5x16	36x44x16	20.2752 M	456.162 kB
Deconvolution	5x5x16	36x44x16	10.1376 M	304.128 kB
Deconvolution	3x3x8	36x44x8	1.8247 M	152.064 kB
Unpooling	2x2	72x88x8	–	253.440 kB
Unpooling	2x2	144x176x8	–	1.1151 MB
Deconvolution	5x5x8	144x176x8	40.5504 M	1.6220 MB
Softmax	N.A.	144x176x14	–	912.384 kB
			91.939 M (Total)	1.622 MB (Max.)

Table 5.6: In detail description of the best network for Nvidia Jetson TX1

As can be seen from the column of selected parameters, all the best networks have dropout layers, unlike networks for micro-controller or FPGA. It shows that if network consists of more layers, dropout layer helps in increasing the accuracy. As explained in section 3.4.1 skip connection helps in transferring the location information, but the results shows that in this case, networks without skip connection gives better performance. Though the accuracy of the network with skip connection is almost similar to those of the top three network, the effect of skip connection can not be confirmed.

Same as micro-controller and FPGA, after 10 generation, 15 optimized networks have been designed for Jetson TX1. The network with best fitness has been explained in details in table 5.6.

The best architecture with filter size and image dimension at each stage has been visualized in figure 5.9.

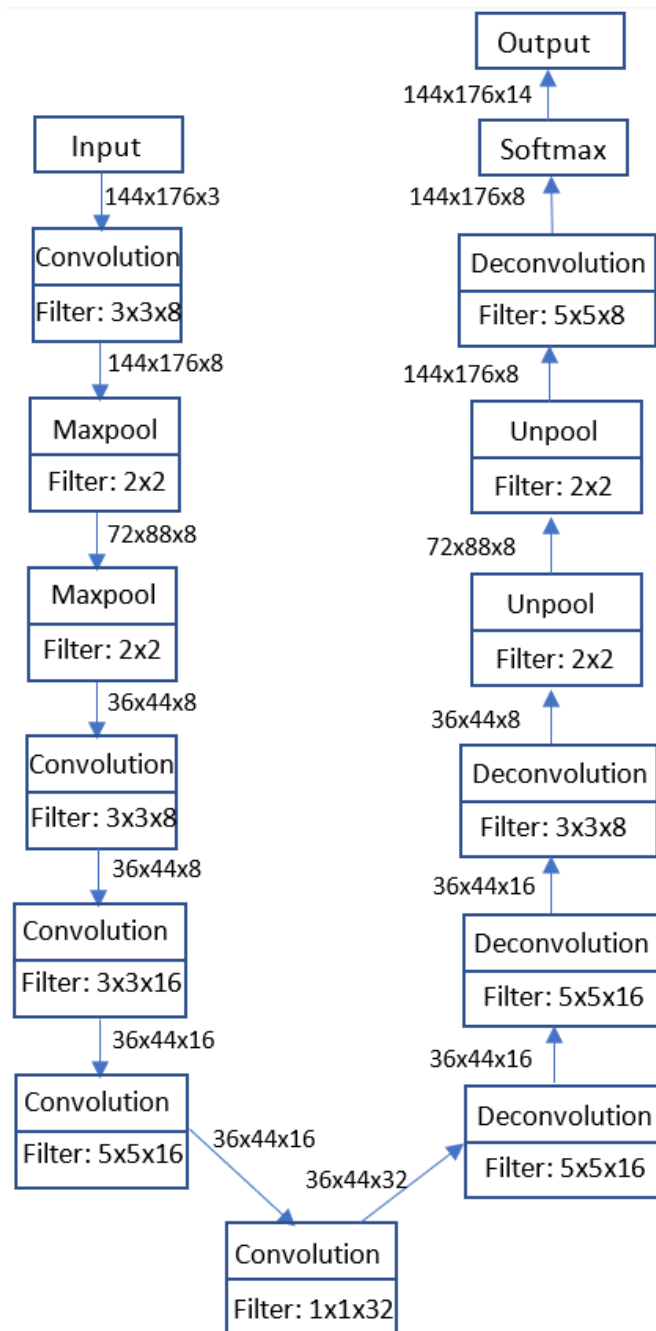


Figure 5.9: Visualization of the best architecture with filter and image size at each step (Jetson TX1)

Segmentation result

Now that we have best suited network, it has been used to segment images which are not used in training. Results of which can be seen in figure 5.10. Results obtained from this network are excellent as can be assumed from the accuracy of the network.

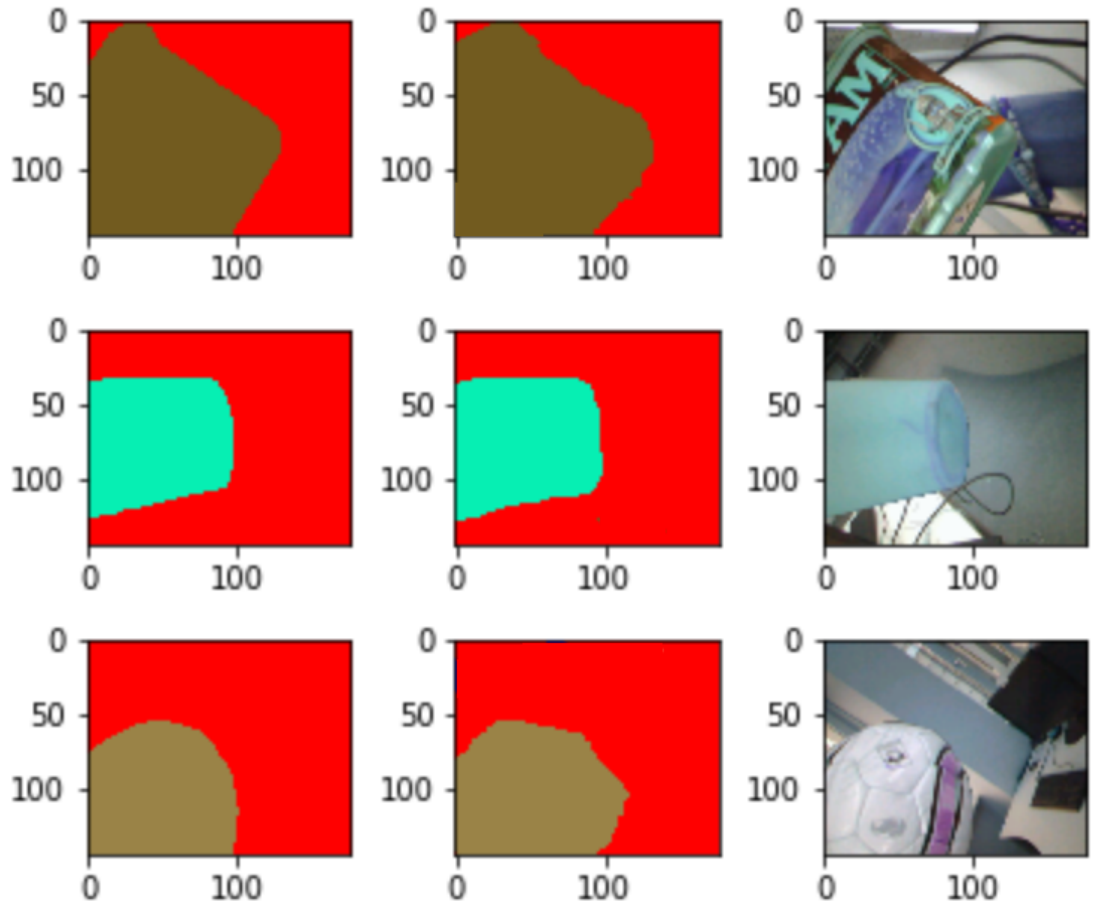


Figure 5.10: Visualization of segmentation Ground truth (left), Predicted output (middle), Image (right) for Jetson TX1

5.2.4 Comparison

In this section, comparison of micro-controller, FPGA and Jetson TX1 for number of operation, memory usage and accuracy has been done.

As can be seen from the table 5.7, for current micro-controller, number of operations and memory usage both are under specified limit. But for the FPGA, memory usage is more than double the allowed limit. Main reason behind it is the dimension of the image. One image with

	Micro-controller		FPGA		Jetson TX1	
	Max. allowed	Best net.	Max. allowed	Best net.	Max. allowed	Best net.
Accuracy	N.A.	0.834	N.A.	0.843	N.A.	0.949
Operations	2M	1.429M	80M	5.125M	60G	3.724G
Memory	0.8MB	0.608MB	0.2MB	0.405MB	2GB	4.05MB

Table 5.7: Comparison of three different hardware regarding resource constraint

144x176 dimension exceeds the memory usage limit. FPGA with more memory or image with less dimension can solve this problem.

As expected, Jetson TX1 gives excellent results because of the available resources. In comparison to allowed resources, network achieves this result with using very less resources.

5.3 Impact of depth Information

Impact of distance information of object on the segmentation accuracy has been done in this work too. At the time of image capturing, depth value for each object has been measured.

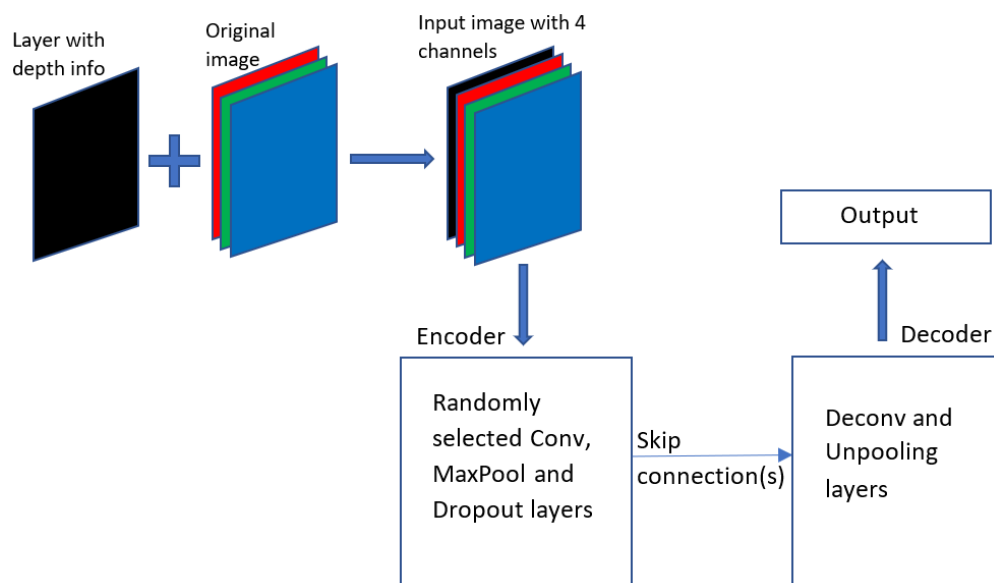


Figure 5.11: Network architecture with depth information

As for using depth information, two different way has been considered. For the first way, depth value can be directly added to the input image, it means that depth value is directly added to the RGB channel of the image. For the second, depth values can be added as an extra channel with input image as shown in figure 5.11. Latter approach has been applied because it gave better results than the former.

	Fitness	Accuracy	Ops. (in M)	Memory (in MB)	Selected Parameters (L_R = Learning rate) (K_S = Kernel size)	Network Layers (Conv[kernel x Feature maps]) (Pool[kernel x channel])
1	2.713	0.849	2.376	0.6082	{'Layers': 9, 'K_S': [1,7], 'Dropout': False, 'Skip': False, 'L_R': 0.001, 'Batch_Size': 4}	Pool4[4 x3] Pool[2x3] Conv [1x5] Conv[1x7] Conv[1x10] Deconv [7x8] Deconv [7x5] Pool[7x5] Unpool[4x5]
2	2.691	0.834	2.447	0.6082	{'Layers': 7, 'K_S': [1,3,7], 'Dropout': False, 'Skip': False, 'L_R': 0.01, 'Batch_Size': 6}	Pool4[4x3] Conv[3x3] Conv[7x5] Conv [1x10] Deconv[3x8] Deconv [1x5] Unpool[4x5]
3	2.544	0.836	3.581	0.6082	{'Layers': 9, 'K_S': [1,3,5,7], 'Dropout': False, 'Skip': True, 'L_R': 0.001, 'Batch_Size': 7}	Pool[2x3] Conv[1x3] Pool[2x3] Conv[1x5] Conv[1x10] Deconv[3x8] Unpool [2x8] Deconv [3x5] Unpool[2x5]
4	2.516	0.834	3.764	0.4055	{'Layers': 9, 'K_S': [1, 3,7], 'Dropout': False, 'Skip': False, 'L_R': 0.02, 'Batch_Size': 4}	Pool[2x3] Conv[1x3] Conv [1x5] Conv [3x8] Conv [1x10] Deconv [7x8] Deconv [3x5] Deconv[7x3] Unpool[2x3]
5	2.375	0.824	4.512	0.8110	{'Layers': 9, 'K_S': [1, 5], 'Dropout': True, 'Skip': True, 'L_R': 0.002, 'Batch_Size': 6}	Pool[2x3] Conv[1x3] Pool[2x3] Pool [2x3] Conv[1x10] Unpool[2x10] Unpool[2x10] Deconv[1x7] Unpool[2x7]
...		...	...	...	...	...
15	1.875	1.875	6.057	0.13178	{'Layers': 5, 'K_S': [1,3,7], 'Dropout': True, 'Skip': False, 'L_R': 0.02, 'Batch_Size': 6}	Conv[1x3] Conv[1x5] Conv[1x8] Conv[1x5] Conv[3x3]

Table 5.8: Best network for Micro-controller with depth information after 10 generations

The extra channel, as shown in figure 5.11, has similar dimension as of the image (height x width). All the pixel of that channel has value equals to the value of depth. This network has been optimized with Evolutionary algorithm for micro-controller. The best suited network after 10 generations have been showed in table 5.8.

When we compare it to result of micro-controller, it can be seen that depth information helps in getting better accuracy. Though it increases the number of operation, memory usage stays under limit for almost all top network. A reason behind not much change in the accuracy can be the resolution of the depth information. Only one depth value for whole image has been measured.

Chapter 6

Conclusion and Future Work

In this thesis, state of the art neural networks for semantic segmentation have been explained and state of the art embedded hardware for neural network have been mentioned. For KIT prosthetic hand, neural network has been designed, taking into consideration available resources. Designed networks can be executed in real-time and for the purpose of semantic segmentation.

6.1 Conclusion

To design the network, state of the art neural network has been taken into consideration and based on that, some of the features of those networks have been applied while finalizing the conceptual design. Training data has been augmented to get descent amount of images and to simulate real situation, background of each image has been also augmented.

Evolutionary algorithm creates optimized network for each embedded hardware according to their constraints. Fitness function for optimization accounts for the speed, accuracy and memory usage. After 10 generation of training with 15 population in each generation, optimized networks have been obtained.

Designed networks for micro-controller and FPGA gives moderate semantic results as can be seen in section 5.2.1 and section 5.2.2. Semantic segmentation needs good amount of computation power when compared to classification. It can be seen from the results that while applying optimization, network tends to go towards less number of operation and thus loses accuracy in process.

For micro-controller, the best optimized network have both the parameter, number of operations and memory usage, under limit. But the optimized network for the FPGA, has more memory usage then allowed. The minimum memory required to run semantic segmentation is more than the available memory of the FPGA.

Network designed for Jetson TX1 gives excellent results (section 5.2.3) with almost no noise and good object boundary detection. It is but expected because the allowed limit for Jetson TX1 resources is too high in comparison to micro-controller and FPGA.

Depth information has been obtained with a Time of Flight (TOF) sensor. It only gives one depth information which is the distance of the object from the hand. It has been used to find out the effect of it on the segmentation accuracy. Though it increases the accuracy, it is not significant to show much improvement on segmentation result.

6.2 Future Work

There are some improvements those can be done to further improve the results.

- **Improved resolution of depth sensor**

The sensor which is currently in use (VL53L1X) can be used to get depth information with 4x4 resolution. That way depth information of different portion of the image can be obtained. These values can be used to generate the depth channel with different values. Which in turn leads to better understanding of the image followed by better learning for the network, thus, better accuracy.

- **Pruning and quantization**

Pruning and quantization has been proved effective in reducing the memory usage without affecting accuracy by too much. It can be applied to trained network to find out how much it affects the memory usage without changing the accuracy.

- **Illumination condition**

While capturing photos, care has been given to the lighting conditions. Images have been captured in the day time only. Image can be captured at different time of the day to make the network more robust. This can also be useful when used in real life condition, which can be any time of the day.

- **Better fitness function**

In this thesis, the weight for each parameters, like number of operations, memory usage and accuracy are same. Which some times leads to discard network with good accuracy but a bit more memory usage or number of operation. This can be taken care by type of fitness function used, but it can also be changed by giving different weights to different parameters.

Bibliography

- [1] Rumelhart, D.E., Williams, R.J., Hinton, G.E.: Learning internal representations by error propagation. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition* 1, 318-362 (1986)
22
- [2] Craven, M.P.: A Faster Learning Neural Network Classifier Using Selective Backpropagation. In: *Proceedings of the Fourth IEEE International Conference on Electronics, Circuits and Systems*, Cairo, Egypt, December 15-18, 1997, vol. 1, pp. 254–258 (1997)
22
- [3] Masters, T. and Land, W.: A new training algorithm for the general regression neural network, In: *IEEE International Conference on Systems, Man, and Cybernetics, Computational Cybernetics and Simulation*, 3 (1997), 1990–1994
23
- [4] Zelinka, I. and Lampinen, J.: An evolutionary learning algorithms for neural networks, In: *5th International Conference on Soft Computing MENDEL'99*, (1999), 410–414.
23
- [5] Mendes, R., Cortez, P., Rocha, M., Neves, J.: Particle swarm for feedforward neural network training. In: *Proceedings of the International Joint Conference on Neural Networks*, vol. 2, pp. 1895–1899 (2002)
24
- [6] <https://towardsdatascience.com/introduction-to-evolutionary>
xi, 29
- [7] Kennedy j, Eberhart RC: Particle swarm optimization. *Proceeding of IEEE International Conference on Neural Networks*; Piscataway, NJ 1995: 1924-1948.
24

- [8] Meissner, M., Schmuker, M., Schneider, G.: Optimized Particle Swarm Optimization (OPSO) and its application to artificial neural network training. *BMC Bioinformatics* 7, 125 (2006)
25
- [9] Le, Lu, Ly, Hai-Bang, Pham, Binh, Le, Vuong, Pham, Tuan, Nguyen, Duy-Hung, Tran, Xuan-Tuan, Le, Tien-Thinh. (2019). Hybrid Artificial Intelligence Approaches for Predicting Buckling Damage of Steel Columns Under Axial Compression. *Materials*. 12. 1670. 10.3390/ma12101670.
xi, 25
- [10] Neural Architecture Optimization, Renqian Luo, Fei Tian, Tao Qin, Enhong Chen, Tie-Yan Liu, arXiv:1808.07233 [cs.LG], 2019
xi, 25, 26
- [11] Yao, X., Liu, Y.: A New Evolutionary System for Evolving Artificial Neural Networks. *IEEE Transactions on Neural Networks* 8(3), 694–713 (1997)
xi, 23, 24
- [12] Hashim, Hashim & Ayinde, Babajide & Abido, Mohammed. (2016). Optimal Placement of Relay Nodes in Wireless Sensor Network Using Artificial Bees Colony Algorithm. *Journal of Network and Computer Applications*. 64. 10.1016/j.jnca.2015.09.013.
xi, 27
- [13] Karaboga, D.: An Idea Based On Honey Bee Swarm For Numerical Optimization, Technical Report-TR06, Erciyes University, Engineering Faculty, Computer Engineering Department (2005)
26
- [14] Basturk, B., Karaboga, D.: An Artificial Bee Colony (ABC) Algorithm for Numeric function Optimization. In: *IEEE Swarm Intelligence Symposium*, May 12-14, 2006, Indianapolis, Indiana, USA (2006)
26
- [15] Design of experiments and focused grid search for neural network parameter optimization, FJ Pontes, GF Amorim, PP Balestrassi, AP Paiva- *Neurocomputing*, 2016 – Elsevier
28
- [16] Random search for hyper-parameter optimization, J Bergstra, Y Bengio - *Journal of machine learning research*, 2012
xi, 28
- [17] Jonas Mockus: On Bayesian Methods for Seeking the Extremum. *Optimization Techniques* 1974: 400-404
28

- [18] Snoek, Jasper; Larochelle, Hugo; Adams, Ryan (2012). "Practical Bayesian Optimization of Machine Learning Algorithms". *Advances in Neural Information Processing Systems*. 28
- [19] Bergstra, James; Bardenet, Remi; Bengio, Yoshua; Kegl, Balazs (2011), "Algorithms for hyper-parameter optimization" , *Advances in Neural Information Processing Systems* 28
- [20] Bain (1873). *Mind and Body: The Theories of Their Relation*. New York: D. Appleton and Company. 14
- [21] McCulloch, Warren; Walter Pitts (1943). "A Logical Calculus of Ideas Immanent in Nervous Activity". *Bulletin of Mathematical Biophysics*. 15
- [22] L. Lai, N. Suda, and V. Chandra. Not all ops are created equal! arXiv preprint arXiv:1801.04326, 2018b. 18, 40
- [23] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, Hartwig Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications" arXiv:1704.04861 (2017) xi, 18
- [24] Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, Quoc V. Le "MnasNet: Platform-Aware Neural Architecture Search for Mobile" arXiv:1807.11626 [cs.CV] (2019) xi, 19, 20
- [25] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, Kurt Keutzer "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size" arXiv:1602.07360 [cs.CV] (2016) xi, 20, 21
- [26] Ariel Gordon, Elad Eban, Ofir Nachum, Bo Chen, Hao Wu, Tien-Ju Yang, Edward Choi "MorphNet: Fast & Simple Resource-Constrained Structure Learning of Deep Networks" arXiv:1711.06798 [cs.LG] xi, 20, 22
- [27] <https://ai.googleblog.com/2019/04/morphnet-towards-faster-and-smaller.html> 21
- [28] <https://github.com/google-research/morph-net> 22
- [29] Nicolas Bernard and Franck Leprevost "Evolutionary Algorithms for Convolutional Neural Network Visualisation" Springer Nature Switzerland AG, E. Meneses et al. (Eds.): CARLA

- 2018, CCIS 979, pp. 18–32, 2019.
30
- [30] Shuxiang Xu and Ling Chen, "A Novel Approach for Determining the Optimal Number of Hidden Layer Neurons for FNN's and Its Application in Data Mining" 5th International Conference on Information Technology and Applications (ICITA 2008)
30
- [31] A. Emin Orhan, Xaq Pitkow, SKIP CONNECTIONS ELIMINATE SINGULARITIES, ICLR 2018
31
- [32] R.K. Srivastava, K. Greff, and J. Schmidhuber. Training very deep networks. NIPS, pp. 2377–2385, 2015.
31, 41
- [33] G. Huang, Z. Liu, K.Q. Weinberger, and L. van der Maaten. Densely connected convolutional networks. 2016. URL: <https://arxiv.org/abs/1608.06993>.
31, 41
- [34] Pavlo M. Radiuk, Impact of Training Set Batch Size on the Performance of Convolutional Neural Networks for Diverse Datasets, Riga Technical university, December 2017, vol. 20, pp. 20–24
32
- [35] M. Takac, A. Bijral, P. Richtarik and N. Srebro, "Mini-Batch Primal and Dual Methods for SVMs," JCMB, 2013
32
- [36] D. R. Wilson and T. R. Martinez, "The General Inefficiency of Batch Training for Gradient Descent Learning," Neural Networks, vol. 16, no. 10, pp. 1429–1451, Dec. 2003. [https://doi.org/10.1016/s0893-6080\(03\)00138-2](https://doi.org/10.1016/s0893-6080(03)00138-2).
32
- [37] Z. Lin, M. Courbariaux, R. Memisevic and Y. Bengio, "Neural networks with Few Multiplications," In Proceedings of the 32d International Conference on Machine Learning, ICML '16, pp. 561–568, 2016.
32
- [38] M. Li, T. Zhang, Y. Chen, and A. J. Smola, "Efficient Mini-Batch Training for Stochastic Optimization," In Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining – KDD '14, 2014. <http://dx.doi.org/10.1145/2623330.2623612>.
32
- [39] P. Weiner, J. Starke, F. Hundhausen, J. Beil, and T. Asfour. The kit prosthetic hand: Design and control. In 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems

- (IROS), pages 3328–3334. IEEE, 2018.
xi, 3, 5, 6
- [40] N. Fukaya, S. Toyama, T. Asfour, and R. Dillmann. Design of the tuat/karlsruhe humanoid hand. In *Intelligent Robots and Systems, 2000.(IROS 2000). Proceedings. 2000 IEEE/RSJ International Conference on*, volume 3, pages 1754–1759. IEEE, 2000.
5
- [41] Felix Hundhausen, Denis Megerle and Tamim Asfour, Resource-Aware Object Classification and Segmentation for Semi-Autonomous Grasping with Prosthetic Hands, *IEEE-RAS 19th International Conference on Humanoid Robots (Humanoids)*, 2019
1, 8
- [42] S. Došen, C. Cipriani, M. Kostic, M. Controzzi, M. C. Carrozza, and D. B. Popovic. Cognitive vision system for control of dexterous prosthetic hands: experimental evaluation. *Journal of neuroengineering and rehabilitation*, 7(1):42, 2010.
6
- [43] G. Ghazaei, A. Alameer, P. Degenaar, G. Morgan, and K. Nazarpour. Deep learning-based artificial vision for grasp classification in myoelectric hands. *Journal of neural engineering*, 14(3):036025, 2017.
7
- [44] Tomović R, Bekey G, Karplus W: A strategy for grasp synthesis with multifingered robot hands. *Proc IEEE Int Conf Robot Autom* 1987, 83-89.
6
- [45] Marko Markovic 1 , Strahinja Dosen, Christian Cipriani, Dejan Popovic, Dario Farina. Stereovision and Augmented Reality for Closed-Loop Control of Grasping in Hand Prostheses. *Journal of Neural Engineering*, Volume 11, Number 4. 2014.
6, 7
- [46] Marković M, Došen S, Popović D B, Graimann B and Farina D 2015 Sensor fusion and computer vision for context-aware control of a multi degree-of-freedom prosthesis *J. Neural Eng.* 12 066022
7
- [47] <https://www.omnisci.com/technical-glossary/embedded-systems>
8
- [48] <https://www.tensorflow.org/lite>
11
- [49] <https://microsoft.github.io/ELL/>
11
- [50] <https://github.com/uTensor/uTensor>
11

- [51] <https://developer.arm.com/ip-products/processors/machine-learning/compute-library>
11
- [52] https://arm-software.github.io/CMSIS_5/NN/html/index.html
11
- [53] <https://github.com/ai-techsystems/deepC>
11
- [54] Wen-Xiong Kang, Qing-Qiang Yang, Run-Peng Liang, The Comparative Research on Image Segmentation Algorithms, First International Workshop on Education Technology and Computer Science, IEEE, 2009.
12, 14
- [55] Rafael C. Gonzalez, and Richard E. Woods, Digital Image Processing, 2nd ed., Beijing: Publishing House of Electronics Industry, 2007
12
- [56] Shi J and Malik.J, "Normalized cuts and image segmentation," IEEE TransMachine Intell, 2000, vol.22, pp.888–905.
13
- [57] Liu Ping, "A survey on threshold selection of image segmentation," Journal of Image and Graphics, 2004, pp.86–92.
13
- [58] Claudio Rosito Jung, "Multiscale image segmentation using wavelets and watersheds," IEEE Symp.2003, Computer Graphics and Image Processing, SIBGRAPI XVI Brazilian Symposium, pp.278–284.
14
- [59] H. Noh, S. Hong, and B. Han, "Learning deconvolution network for semantic segmentation," in ICCV, pp. 1520–1528, 2015.
x, xi, 34, 35, 36
- [60] S. Ji, W. Xu, M. Yang, and K. Yu. "3D convolutional neural networks for human action recognition", TPAMI, 35(1):221–231, 2013.
34
- [61] K. Simonyan and A. Zisserman. "Two-stream convolutional networks for action recognition in videos", In NIPS, 2014.
34
- [62] K. Simonyan and A. Zisserman. "Very deep convolutional networks for large-scale image recognition", In ICLR, 2015.
35, 36

- [63] V. Badrinarayanan, A. Handa, and R. Cipolla, “Segnet: A deep convolutional encoder-decoder architecture for robust semantic pixel-wise labelling,” arXiv preprint arXiv:1505.07293, 2015.
x, xi, 35, 36, 37
- [64] Adam Paszke, Abhishek Chaurasia, Sangpil Kim, Eugenio Culurciello, “ENet: A Deep Neural Network Architecture for Real-Time Semantic”, arXiv:1606.02147v1 [cs.CV], 2016.
x, 36
- [65] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” arXiv preprint arXiv:1512.03385, 2015.
xii, 37
- [66] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in MICCAI, pp. 234–241, Springer, 2015
x, xi, xii, 15, 38
- [67] <https://medium.com/@sdoshi579/convolutional-neural-network>
xi, 17